

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA Informatică Română

LUCRARE DE LICENȚĂ

**Detectarea automată a numerelor de
înmatriculare folosind tehnici clasice
de viziune artificială și clasificatori
statistici**

Conducător științific
Conferențiar, Dr. Bufnea Darius

Absolvent
Pușcaș Raul-Andrei

ABSTRACT

Recunoașterea automată a numerelor de înmatriculare reprezintă procesul de extragere și recunoaștere a caracterelor care formează identificatorul unic al unui vehicul.

Sistemul de recunoaștere captează imaginea prin intermediul unei camere (B/W, Infraroșu etc), pe care o trimite mai departe unei unități de procesare care execută un algoritm format din 3 etape: localizarea numărului de înmatriculare, segmenatarea caracterelor, recunoașterea caracterelor. Ulterior, informația extrasă poate fi folosită în cadrul unor servicii precum: securizarea parcarilor, monitorizarea traficului, etc.

Comparat cu abordările existente ale problemei soluția prezentată în această lucrare se evidențiază prin reziliența pe care o oferă față de diferiții factori care pot afecta calitatea sistemului: condiții de lumină variabile, diferite formate ale plăcuței de înmatriculare, reziduuri care pot obscura caracterele. Dar și prin nivelul ridicat de optimizare adus întregului proces.

Cuprins

1	Introducere	1
2	Metode de localizare a obiectelor	3
2.0.1	Metode bazate pe caracteristici cromatice	3
2.0.2	Metode bazate pe conturul obiectelor	5
2.0.3	Metode bazate pe textura obiectelor	9
3	Segmentarea imaginii	12
3.0.1	Conectivitatea pixelilor	12
3.0.2	Analiza proiecțiilor	12
3.0.3	Utilizarea informațiilor apriori privind aspectul caracterelor .	14
3.0.4	Analiza conturului caracterelor	15
4	Recunoașterea caracterelor	17
4.1	Potrivire de șabloane	17
4.2	Clasificatori statistici	18
4.2.1	Mașini pe suport vectorial	19
4.2.2	Rețele neuronale artificiale	24
5	Implementarea sistemului IORA	28
5.1	Arhitectura sistemului	28
5.1.1	Diagrama de componente	29
5.1.2	Actori și cazuri de utilizare	29
5.1.3	Diagrama de clase	30
5.1.4	Decizii de proiectare	31
5.2	Tehnologii	32
5.2.1	Nucleu	32
5.2.2	Nivelul de aplicație	32
5.3	Detalii tehnice	33
5.3.1	Modulul de intrare (Input)	33
5.3.2	Fluxul de procesare	35

5.3.3	Nivelul de aplicație	39
5.4	Seturi de date	40
5.4.1	Setul de evaluare	41
5.4.2	Setul de antrenare a clasificatorului SVM	42
5.5	Testare și metrice	44
5.6	Experimente	47
5.6.1	Îmbunătățiri progresive ale fluxului de recunoaștere	49
6	Concluzii	55
	Bibliografie	56

Capitolul 1

Introducere

Creșterea constantă a numărului de vehicule aflate în circulație a făcut ca identificarea automată a acestora să devină o componentă esențială în infrastructura modernă de transport. Lucrarea de față propune un sistem de recunoaștere automată a numerelor de înmatriculare (ALPR — Automatic License Plate Recognition) construit pe o arhitectură modulară, bazată exclusiv pe tehnici clasice de viziune artificială pentru etapele de procesare a imaginii și pe un clasificator statistic de tip SVM (Support Vector Machine) pentru recunoașterea caracterelor.

Studiul a fost motivat de cerințele concrete ale unui sistem de control al accesului auto într-o parcare situată pe teritoriul României. Acest context impune două constrângeri determinante. În primul rând, sistemul trebuie să recunoască fiabil o varietate largă de formate: plăcuțe uzuale –două litere –două cifre –trei litere, plăcuțe provizorii cu serie roșie, plăcuțe pentru autoturisme noi în format extins, plăcuțe diplomatice (CD, TC, CO), plăcuțe ale Ministerului Afacerilor Interne și plăcuțe străine ale autovehiculelor aflate în tranzit. În al doilea rând, întregul flux de procesare, de la achiziția cadrului până la validarea numărului trebuie să aibă un timp de execuție redus, de ordinul sutelor de milisecunde, astfel încât bariera de acces să fie acționată în cel mult 1–2 secunde de la momentul sosirii vehiculului.

Soluția propusă structurează procesul în trei etape secvențiale. Etapa de *localizare* izolează regiunea plăcuței într-o imagine de intrare folosind operatorul Sobel pentru extragerea contururilor verticale, urmat de operații morfologice și filtre geometrice care elimină regiunile candidate necorespunzătoare. Etapa de *segmentare* separă caracterele individuale prin proiecții orizontale și verticale ale imaginii binarizate. În final, etapa de *recunoaștere* clasifică fiecare caracter segmentat printr-un model SVM multi-clasă antrenat pe un set de caractere etichetate. Întreg ansamblul este implementat fără a recurge la rețele neuronale profunde, ceea ce permite execuția pe hardware modest, fără accelerare GPU.

În comparație existente, soluția prezentată se evidențiază prin două contribuții. Prima este reziliența la variațiile de mediu și de format precum diferențe de lumi-

nozitate, fonturi neomogene între seriile de plăcuțe, dimensiuni variabile și ocluzii parțiale, caracteristică obținută prin alegerea unor tehnici robuste în fiecare etapă a fluxului. A doua contribuție este nivelul de optimizare adus fiecărui modul, astfel încât sistemul să respecte constrângerea de timp real pe o platformă embedded.

Lucrarea este organizată în șase capitole. Primele trei capitole au caracter teoretic și prezintă fundamentele necesare. Capitolul 2 tratează problema localizării obiectelor de interes într-o imagine și introduce, ca aplicație particulară, metodele folosite în literatură pentru localizarea plăcuței de înmatriculare. Capitolul 3 prezintă tehnicile de segmentare a regiunilor identificate, cu accent pe separarea caracterelor. Capitolul 4 discută clasificarea caracterelor, pornind de la metodele bazate pe șabloane și culminând cu fundamentele teoretice ale clasificatorului SVM. Capitolul 5 constituie contribuția practică a lucrării: descrie arhitectura, deciziile de implementare și evaluarea experimentală a sistemului IORA, inclusiv metricile obținute pe setul de testare și rezultatele obținute în condiții reale de operare. Capitolul 6 sintetizează concluziile, evidențiază limitările soluției actuale și schițează direcțiile de dezvoltare ulterioară.

Capitolul 2

Metode de localizare a obiectelor

Detectarea unui obiect de interes într-o imagine este una dintre cele mai abordate teme din viziunea artificială. Pentru a atinge acest scop variantele sunt numeroase, toate având o caracteristică comună: dependența de trăsăturile obiectului vizat. Un exemplu practic îl reprezintă detecția fețelor în imagini: pentru ca o regiune să fie identificată drept față, aceasta trebuie să prezinte o formă aproximativ ovală, în interiorul căreia să se regăsească două zone simetrice corespunzătoare ochilor, dispuse în treimea superioară, o structură verticală alungită asociată nasului, plasată central, și o regiune orizontală sub aceasta, corespunzătoare gurii. Fiecare obiect poate fi astfel caracterizat printr-un set de trăsături distinctive geometrice, cromatice sau texturale care permit diferențierea sa de restul scenei.

În contextul lucrării de față, obiectul de interes îl reprezintă plăcuța de înmatriculare, a cărei localizare precisă constituie o secțiune critică a întregului proces. Regiunea determinată de numărul de înmatriculare poate fi localizată folosind diferite metode care pot fi clasificate în funcție de caracteristica utilizată [1]: culoare, textură, sau contur. Ultima categorie este întâlnită cel mai frecvent în aplicații și se bazează pe presupunerea că în interiorul plăcuței există schimbări de intensitate mai drastice și mai dese decât în oricare altă zonă a imaginii. Metodele bazate pe culoare au ca fundație presupunerea că există o distincție clară între culoarea numărului de înmatriculare și fundal; iar cele pe textură se folosesc de prezența caracterelor în interiorul plăcuței pentru a delimita regiunea acestuia.

2.0.1 Metode bazate pe caracteristici cromatice

Această categorie de metode vizează obiectele care prezintă în interiorul lor o paletă cromatică bine definită și ușor diferențiabilă de fundalul imaginii. Un exemplu clasic îl reprezintă localizarea semnelor de circulație, a căror culoare standardizată permite separarea lor de restul scenei.

Punctul de plecare al acestor metode îl constituie alegerea unui model croma-

tic. Un model cromatic reprezintă un model matematic abstract care descrie modul în care culorile pot fi reprezentate într-un spațiu cu mai multe dimensiuni, cel mai frecvent tridimensional. Modelul RGB (Red, Green, Blue) descrie fiecare culoare prin combinația aditivă a trei componente de bază și corespunde modului în care senzorii camerelor digitale achiziționează imaginea. Dezavantajul său major în contextul localizării este că informația despre culoare și cea despre luminozitate sunt amestecate în cele trei canale: aceeași culoare percepută își modifică semnificativ valorile RGB atunci când condițiile de iluminare se schimbă. Din acest motiv, abordările bazate pe culoare convertesc de regulă imaginea într-un model din familia HSV/HSL/HSI (Hue, Saturation, Value/Lightness/Intensity), în care nuanța (*hue*) este separată de saturație și de componenta de luminozitate. Această separare conferă o anumită invarianță la variațiile de iluminare, deoarece nuanța unei culori rămâne relativ stabilă chiar și atunci când intensitatea luminii se modifică.

În cadrul sistemelor ALPR, numeroase lucrări au valorificat caracteristicile cromatice ale plăcuței pentru a o localiza [2], [3].

Prima lucrare [2] tratează localizarea numerelor de înmatriculare din China, unde există un număr restrâns de combinații standardizate de culori (de exemplu, caractere albe pe fundal albastru pentru autoturisme sau caractere negre pe fundal galben pentru vehiculele de mare tonaj). Ideea de bază este că o anumită combinație culoare-fundal apare aproape exclusiv în regiunea plăcuței. Trecerea la modelul HLS rezolvă problema robusteții, dar introduce un cost de calcul prohibitiv: conversia RGB $\rightarrow$ HLS devine punctul critic, necesitând aproximativ o secundă pentru o imagine de 1024×768 pixeli. Soluția finală păstrează insensibilitatea la iluminare a modelului HLS, dar evită conversia propriu-zisă: pixelii sunt clasificați în 13 categorii definite direct în domeniul RGB, în funcție de variația luminozității. Regiunea plăcuței este apoi delimitată prin analiza histogramelor pe direcțiile verticală și orizontală și validată pe baza raportului lățime/înălțime (WHR). Sistemul atinge o rată de citire corectă de 90 % din imagini, în mai puțin de 0,3 secunde.

A doua lucrare [3] abordează plăcuțele din Taiwan, care utilizează patru culori distincte (alb, negru, roșu și verde). De această dată imaginea RGB este transformată efectiv în modelul HSI (Hue, Saturation, Intensity), pe baza căruia se construiește un detector de contururi cromatice. Spre deosebire de un detector de margini clasic, acesta reacționează doar la cele trei tipuri de tranziții întâlnite în interiorul plăcuței — negru-alb, roșu-alb și verde-alb —, ignorând restul contururilor din imagine. Pentru fiecare dintre hărțile de nuanță, saturație, intensitate și contur se calculează câte o hartă *fuzzy*, ale cărei valori exprimă gradul în care fiecare pixel aparține unei plăcuțe. Cele patru hărți sunt apoi combinate printr-un agregator fuzzy în două etape, iar regiunile cu valori maxime locale și dimensiune suficientă sunt reținute drept candidați. Folosind exclusiv informația de localizare, metoda atinge o rată de



(a) Localizarea zonei de interes generale folosind proiecții verticale



(b) Extragerea numărului de înmatriculare folosind proiecții verticale



(c) Numărul de înmatriculare extras

Figura 2.1: Procesul de extracție al numărului de înmatriculare [2]

succes de 97,9 %, respectiv 93,7 % pentru întregul sistem de recunoaștere.

Această categorie de metode prezintă atât avantaje, cât și limitări importante. Pe de o parte, atunci când contextul problemei garantează o paletă cromatică standardizată, ele oferă o acuratețe ridicată și un timp de execuție redus, datorită algoritmilor relativ simpli și ușor de optimizat. Pe de altă parte, ele nu generalizează la un spectru larg de formate, fiind dependente de un set restrâns de combinații de culori cunoscute *a priori*; din acest motiv, lucrările menționate vizează țări precum China sau Taiwan, unde variațiile sunt limitate. În plus, performanța lor rămâne sensibilă la condițiile de iluminare și la degradarea cromatică a plăcuțelor reale.

2.0.2 Metode bazate pe conturul obiectelor

Această abordare are la bază presupunerea că în interiorul plăcuței există o densitate ridicată de linii verticale și orizontale care fac regiunea respectivă să iasă în evidență. Indiferent de abordarea ulterioară, fluxul de procesare începe întotdeauna de la transformarea imaginii în tonuri de gri. În fotografie și viziune pe calculator, o imagine în tonuri de gri este o imagine în care fiecare pixel este o nuanță de

gri, unde valoarea fiecărei celule este un singur eșantion, ținând doar informația intensității. Această caracteristică a imaginilor în tonuri de gri reprezintă un factor important, deoarece în cadrul trecerilor, mai exact la marginile unui obiect apar întotdeauna diferențe mari de intensitate, cu atât mai mult în cadrul unui număr de înmatriculare unde trecerile sunt în general de la alb la negru, roșu la alb, verde la roșu, etc. Treceri bruște, clare și evidente într-o imagine convertită la alb-negru.

În [4] asupra imaginii convertite este aplicat operatorul Sobel pentru a extrage atât marginile verticale cât și cele orizontale. Operatorul Sobel constă în 2 kernel-uri specializate pentru detectarea liniilor convoluționate asupra întregii imagini pentru a produce un rezultat care evidențiază marginile obiectelor, trecerile bruște de la o intensitate la alta. Rezultatul intermediar generat de operatorul Sobel este binarizat folosind diferiți algoritmi de prag precum metoda histogramei, grad4 etc. Pentru o localizare robustă sunt aplicate imaginea *skip* și imaginea integrală, prima având ca scop calcularea, pentru un dreptunghi de dimensiune predefinită, a distanței dintre pixelii albi și cei negri, iar imaginea integrală calculează pentru fiecare punct suma cumulativă a intensităților tuturor pixelilor aflați în regiunea superioară și stângă. Folosind această metodă, autorii au reușit să obțină o rată de localizare cu succes a numărului de înmatriculare de 99,95 % cu un timp mediu de procesare de 47 ms.

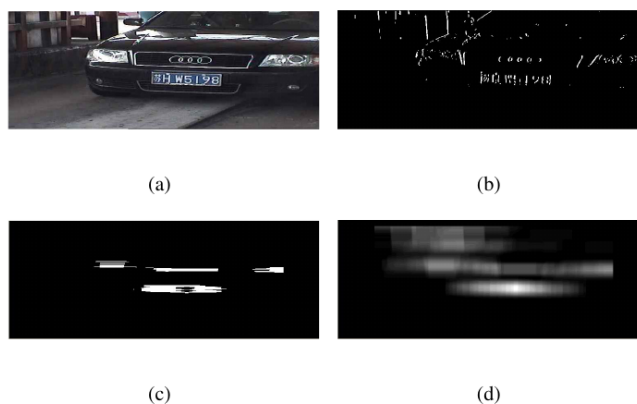
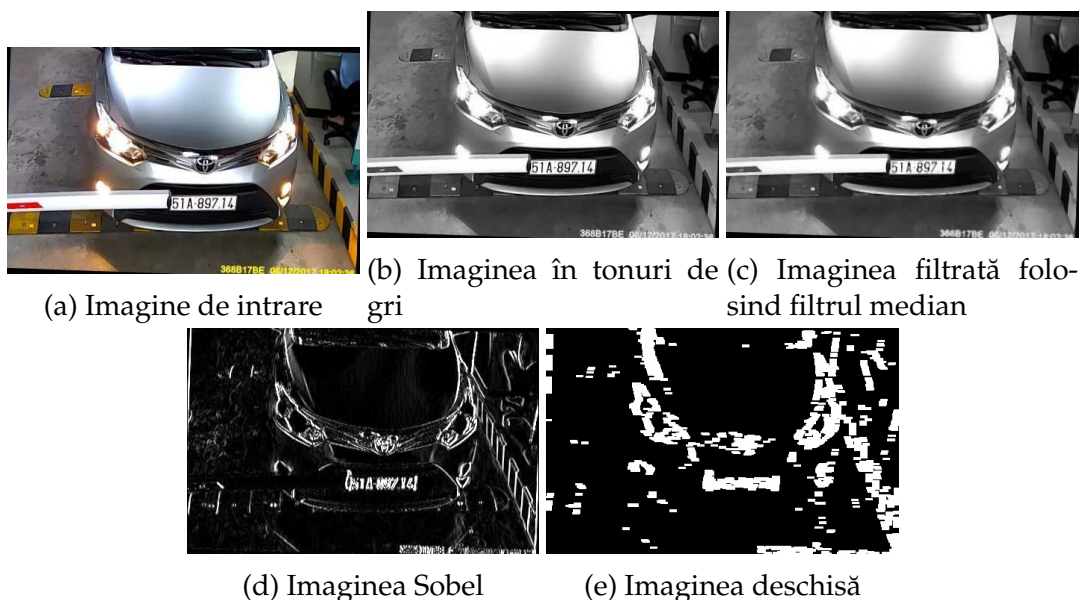


Figura 2.2: (a) Imaginea RGB inițială, (b) Imaginea binară a marginilor, (c) Imaginea *skip*, (d) Imaginea integrală [4]

Similar cu [4], [5] utilizează operatorul Sobel ca componentă primară pentru extragerea marginilor și a conturilor, totuși deosebirile apar încă din etapa de pre-procesare a imaginii (etapa de pre-procesare este formată din întreaga suită de pași aplicată imaginii înainte ca algoritmul principal de detectare a conturilor să fie rulat). [5] se folosesc de filtrul median pentru a atenua zgomotul imaginii. În viziunea pe calculator, atenuarea zgomotului presupune crearea unei funcții care capturează informațiile și tiparele imaginii lasând la o parte detaliile la scară redusă sau alte variații rapide de intensitate.

Pe lângă filtrarea mediană, pentru a asigura un rezultat robust, autorii definesc o mască de dimensiuni predefinite preluate din condiționarea problemei asupra căreia să continue cu următoarele etape ale procesului, respectiv operatorul Sobel, binarizarea imaginii și operația morfologică de deschidere. Aceasta din urmă constă în aplicarea consecutivă a două transformări fundamentale morfologice: eroziune și dilatare.

Operația de eroziune are rolul de a diminua obiectele din prim-plan și de a elimina structurile izolate care nu aparțin plăcuței. Aceasta presupune aplicarea unui kernel asupra întregii imagini, pixelul central fiind setat ca alb dacă toți pixelii acoperiți de kernel sunt la rândul lor albi. Operația de dilatare reprezintă opusul celei de eroziune, este formată din același algoritm, însă structura sa decizională transformă pixelul evaluat în „alb” dacă cel puțin un pixel acoperit de elementul structurant este activ.



Deși Sobel oferă rezultate bune, costul său din punct de vedere computațional a motivat autorii lucrării [6] să dezvolte un nou algoritm pentru detectarea marginilor verticale. Metoda propusă omite în totalitate marginile orizontale din cadrul unei imagini, axându-se strict pe cele verticale. Această decizie a fost fundamentată pe observația că marginile orizontale pot cauza diferite elemente din compoziția mașinii precum grilajul să fie identificate în mod eronat drept numere de înmatriculare.

Fluxul de procesare propus în [6] diferă față de cele menționate anterior. Inițial, asupra imaginii greyscale îi este aplicat un algoritm de binarizare cu prag adaptiv.

Acesta este urmat de ULEA (Unwanted Line Elimination Algorithm), un procedeu specializat de eliminare a artefactelor generate de către binarizare și a liniilor verticale nedorite de o lățime predefinită.

În final este aplicat VEDA, propus ca înlocuitor al operatorului Sobel de către autori. Această metodă constă în convoluționarea unui kernel de 2×4 asupra întregii imagini și o structură decizională care selectează un pixel ca fiind activ sau inactiv pe baza unor reguli logice în locul operațiilor clasice de convoluție.

Prin renunțarea la procesarea liniilor orizontale și optimizarea mecanismului de detecție, algoritmul propus are o viteză medie de rulare de 15 ms, o îmbunătățire semnificativă în comparație cu operatorul Sobel, care atinge o viteză medie de 130 ms.

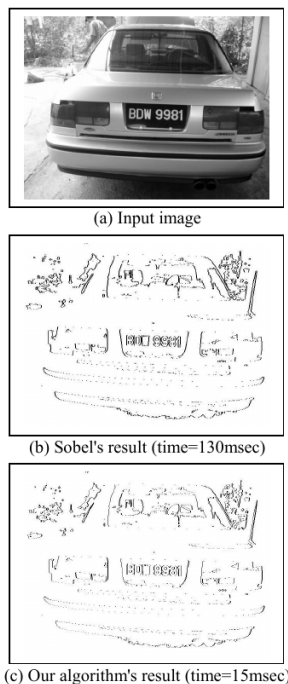


Figura 2.4: Comparație între rezultatele generate de Sobel și VEDA [6]

Datorită faptului că [4], [5], [6] folosesc drept componentă principală operatorul Sobel, respectiv VEDA, aceste abordări sunt capabile să localizeze plăcuțe conținând numere de înmatriculare aflate în imagine la o înclinație de până la 15° . Pentru a combate acest lucru, în [7] este folosit operatorul Hough drept selector principal.

Spre deosebire de operatorii de gradient (Sobel sau VEDA), care se bazează pe analiza locală a pixelilor, transformarea Hough reprezintă o tehnică de extracție a caracteristicilor utilizată pentru a identifica forme geometrice prin procesul de votare într-un spațiu al parametrilor. În contextul localizării numerelor de înmatriculare, aceasta este utilizată preponderent pentru detecția liniilor drepte care definesc conturul dreptunghiular al plăcuței.

Principiul fundamental constă în maparea fiecărui punct de margine (x, y) din spațiul imaginii în spațiul polar (ρ, θ) , utilizând ecuația:

$$\rho = x \cos(\theta) + y \sin(\theta)$$

unde ρ reprezintă distanța perpendiculară de la origine la dreaptă, iar θ este unghiul dintre axa x și această perpendiculară.

Implementarea transformării Hough în detrimentul implementării operatorului Sobel sau VEDA aduce anumite avantaje din punct de vedere al fiabilității sistemului, acesta fiind capabil să localizeze în mod corect plăcuțe de înmatriculare înclinate la 30° comparat cu 15° . De asemenea, transformarea Hough nu este atât de sensibilă la variațiile de intensitate ale imaginii, astfel că o imagine de calitate nu este imperios necesară.

Totuși, performanța sporită vine cu un cost de complexitate computațională ridicat. Deoarece procesul de votare trebuie efectuat pentru fiecare pixel de margine și pentru o gamă largă de unghiuri, consumul de memorie (pentru matricea acumulator) și timpul de procesare sunt semnificativ mai mari față de un operator de convoluție simplu precum Sobel. Din acest motiv, în sistemele de timp real, transformarea Hough este adesea aplicată pe o regiune de interes (ROI) restrânsă sau este precedată de o etapă de binarizare agresivă pentru a reduce numărul de pixeli procesați.

2.0.3 Metode bazate pe textura obiectelor

Metodele bazate pe textură pornesc de la presupunerea că în interiorul numerelor de înmatriculare prezența caracterelor oferă o textură specifică și ușor diferențiabilă în comparație cu fundalul/ restul elementelor prezente în imagine.

În literatura de specialitate, există multiple abordări ale acestei categorii de metode, [8], [9], care se diferențiază prin modul în care este descrisă textura caracterelor și prin tehnica folosită pentru a evidenția regiunile cu densitate ridicată de tranziții.

În [8] autorii aleg drept caracteristică definitorie a texturii densitatea de linii verticale prezente în cadrul numărului de înmatriculare. Aceștia propun un sistem al cărui prim pas este scăderea cromatică (*color subtraction*), un algoritm ce primește ca parametri de intrare imaginea achiziționată anterior și o listă de elemente dintr-unul dintre modelele cromatice posibile; acest algoritm va păstra activi din imagine doar pixelii ale căror valori apar în lista primită drept parametru de intrare, iar pe restul îi va dezactiva. Următorul pas este aplicarea operatorului Sobel pentru a evidenția liniile verticale. În continuare va fi construită o hartă de densitate ponderată (*weight-based density map*) folosind următorul algoritm de asignare a greutăților, în care

creșterea, respectiv descreșterea unei greutateți asignate este direct proporțională:

$$W_{t_{\text{decr}}} \propto (i - p) \quad (2.1)$$

$$W_{t_{\text{decr}}} \propto \gamma \quad (2.2)$$

$$W_{t_{\text{decr}}} = \partial \gamma (p - i) \quad (2.3)$$

$$W_t = \sum_{i=0}^n t_w (W_t + (px_i)\gamma + (1 - px_i)(\partial\gamma(p - i))) \quad (2.4)$$

$$t_w = \frac{W_t - |W_t|}{-2W_t} \beta + \frac{W_t + |W_t|}{2W_t} \quad (2.5)$$

unde ∂ este constanta de decădere (*decay constant*), γ este factorul de incrementare a greutateții, β greutatea inițială, n numărul de puncte de coordonate, i coordonata muchiei curente, p coordonata muchiei anterioare, iar W_t valoarea greutateții.

După finalizarea construcției hărții de densitate putem identifica posibilele regiuni în care apare un număr de înmatriculare.

Spre deosebire de [8], care descrie textura prin densitatea liniilor verticale, în [9] autorii pornesc de la ideea că plăcuța poate fi privită ca o regiune în care textura imaginii prezintă o „neregularitate” locală pronunțată, mai exact treceri bruște și dese între caracterele întunecate și fundalul deschis. Pentru a evidenția aceste zone, ei propun o tehnică de segmentare adaptivă numită SCW (Sliding Concentric Windows), care nu măsoară direct contururile, ci variația statistică a intensităților dintr-o vecinătate.

Metoda se bazează pe utilizarea a două ferestre dreptunghiulare concentrice, una interioară A de dimensiune $(2X_1) \times (2Y_1)$ și una exterioară B de dimensiune $(2X_2) \times (2Y_2)$, care parcurg împreună întreaga imagine, pixel cu pixel, pornind din colțul stânga-sus. Pentru fiecare poziție se calculează în cele două ferestre câte o măsură statistică M (media sau deviația standard a intensităților), iar pixelul central este clasificat în funcție de raportul dintre cele două măsuri:

$$I_{\text{AND}}(x, y) = \begin{cases} 0, & \text{dacă } \frac{M_B}{M_A} \leq T \\ 1, & \text{dacă } \frac{M_B}{M_A} > T \end{cases}$$

unde T este un prag stabilit experimental. Atunci când raportul depășește pragul, vecinătatea pixelului prezintă o variație texturală suficient de mare pentru ca acesta să fie considerat parte a unei regiuni de interes. Parametrii X_1, X_2, Y_1, Y_2 se aleg astfel încât raportul de aspect al ferestrelor să fie apropiat de cel al obiectelor căutate (caracterele plăcuței).

Imaginea binară I_{AND} obținută prin SCW este apoi combinată cu imaginea originală printr-o operație logică ȘI, rezultatul fiind binarizat cu ajutorul metodei de prag

adaptiv Sauvola, în care pragul fiecărui pixel depinde de media și deviația standard locale dintr-o fereastră de dimensiune $b \times b$:

$$T(x, y) = m(x, y) \left[1 + k \left(\frac{\sigma(x, y)}{R} - 1 \right) \right]$$

Asupra imaginii binarizate se aplică o analiză a componentelor conexe (CCA), iar obiectele rezultate sunt filtrate pe baza unor criterii geometrice: raportul de aspect, orientarea (sub 35°) și numărul Euler. Acesta din urmă, definit ca diferența dintre numărul de obiecte și numărul de „găuri” interioare, este folosit pe baza observației că o plăcuță validă conține cel puțin patru caractere, deci minimum trei goluri ($E > 3$); regiunile care nu îndeplinesc aceste condiții sunt eliminate. Pentru a trata și plăcuțele cu fundal închis și caractere deschise, dacă nicio regiune candidată nu este găsită, imaginea este inversată și întregul proces reia.

Etapa de localizare este urmată de segmentarea caracterelor, realizată tot prin SCW aplicat regiunii decupate, și de recunoașterea propriu-zisă cu o rețea neuronală probabilistică (PNN) cu topologia 108-180-36. Testat pe un set de 1334 de imagini din scene naturale, cu fundaluri și condiții de iluminare variate, sistemul a segmentat corect plăcuța în 96,5 % din cazuri, recunoașterea întregii plăcuțe atingând 89,1 %, respectiv o rată globală de succes de 86 %.

Capitolul 3

Segmentarea imaginii

Segmentarea imaginii joacă un rol crucial în sistemele ALPR. Procesul presupune divizarea imaginii într-un anumit număr de segmente informative, sau regiuni care sunt mai simple de studiat.

În literatura de specialitate, pentru a segmenta numărul de înmatriculare sunt propuse mai multe tehnici [1]:

- Analiza conectivității pixelilor [10]
- Analiza proiecțiilor
- Utilizarea informațiilor apriori privind aspectul caracterelor
- Analiza conturului caracterelor

3.0.1 Conectivitatea pixelilor

Această abordare presupune binarizarea regiunii de interes extrase de modulul anterior, urmată de analiza componentelor conexe (CCA) și de aplicarea unei structuri decizionale pentru filtrarea regiunilor identificate de CCA. Selecția se bazează în principal pe criterii precum similitudinea dimensiunilor și a proporțiilor zonelor identificate de CCA.

Principalul dezavantaj al acestei metode este sensibilitatea la rotație, selecția fiind realizată pe baza uniformității geometrice a regiunilor, numerele de înmatriculare rotite sau înclinate parțial sunt adesea segmentate în mod incorect.

3.0.2 Analiza proiecțiilor

Contrastul ridicat dintre fundalul plăcuței și caractere face ca, în urma binarizării, cele două să fie reprezentate prin valori opuse. Această proprietate stă la

baza unei familii largi de metode de segmentare, care exploatează distribuția pixelilor prin combinarea analizei proiecțiilor verticale și orizontale.

O proiecție se obține prin construirea unei histograme a densității pixelilor activi de-a lungul uneia dintre cele două axe de coordonate, un pixel fiind considerat activ atunci când valoarea sa aparține unui interval predefinit. În cazul, frecvent întâlnit în literatura de specialitate, al imaginilor binarizate, pixelii activi iau una dintre valorile 0 sau 255.

Un exemplu reprezentativ este sistemul propus în [7], în care extracția numărului de înmatriculare se bazează pe transformarea Hough pentru corectarea variațiilor de înclinare și rotație.

Segmentarea începe cu analiza proiecțiilor orizontale, prin care numărul de înmatriculare este divizat pe rânduri. Etapa este esențială pentru plăcuțele dispuse pe mai multe rânduri (precum cele ale motocicletelor), întrucât separarea pe rânduri normalizează proporțiile și dimensiunea relativă a caracterelor în raport cu sub-segmentul care le conține. În absența ei, o evaluare globală ar introduce discrepanțe majore: pe un singur rând un caracter ocupă aproximativ 85% din înălțimea imaginii, în timp ce pe o plăcută cu două rânduri ajunge la cel mult 50%.

După divizarea pe rânduri, proiecțiile verticale sunt utilizate pentru a determina pozițiile de început și de final ale fiecărui caracter. Segmentele astfel obținute sunt apoi filtrate printr-un set de constrângeri, menite să elimine elementele segmentate eronat drept caractere.

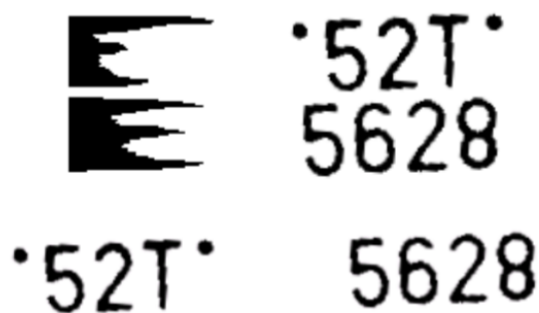


Figura 3.1: Rezultatul divizării pe rânduri [7]

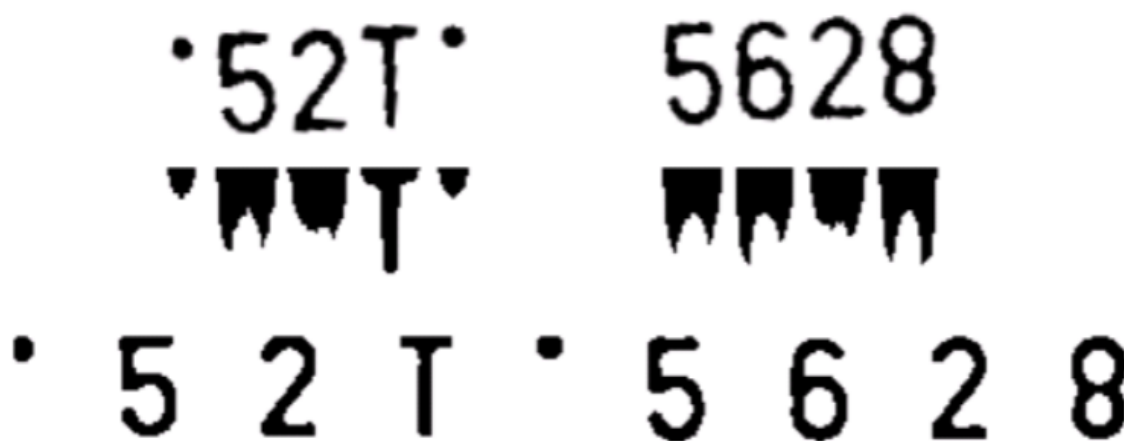


Figura 3.2: Rezultatul segmentării folosind proiecții verticale [7]

3.0.3 Utilizarea informațiilor apriori privind aspectul caracterelor

Cunoștințe apriori în legătură cu aspectul caracterelor, font-ul, mărimea și proporțiile lor pot îmbunătăți în mod semnificativ procesul de segmentare.

În [11] este propusă o abordare similară cu cea prezentată anterior în [7] în care numărul de înmatriculare este adus la o mărime și înclinare standard, iar etapa de segmentare este realizată folosind proiecții orizontale și verticale. Spre deosebire de metoda amintită în mod anterior, autorii se folosesc pe parcursul etapei de segmentare de o structură decizională care va ajuta la eliminarea rezultatelor fals-pozitive.

Particularitatea acestei abordări constă în combinarea informației furnizate de proiecția verticală cu cea obținută printr-o etapă preliminară de analiză a componentelor conexe (pre-extragerea zonelor conexe). Rolul acestei etape preliminare este de a atenua efectul zgomotului asupra proiecției: în prezența zgomotului, profilul de proiecție al imaginii binarizate prezintă punți (conexiuni) între caractere, care îngreunează separarea lor. Prin filtrarea prealabilă a zonelor conexe, numărul acestor conexiuni din profilul de proiecție se reduce semnificativ, motiv pentru care varianta astfel curățată este reținută pentru pașii următori, în detrimentul proiecției obținute direct din imaginea zgomotoasă.

Pe lângă atenuarea zgomotului, zonele conexe pre-extrase furnizează și informațiile apriori care stau la baza extragerii: lățimea caracterelor și poziția lor în cadrul plăcuței. Conform specificului plăcuțelor de înmatriculare avute în vedere, toate caracterele au aceeași lățime, care poate fi estimată direct din lățimea unei zone conexe deja izolate corect (de exemplu cifrele 5 sau 0). Se definește poziția centrală a unui caracter ca fiind poziția orizontală a marginii sale stângi la care se adaugă jumătate din lățimea caracterului, iar distanța dintre pozițiile centrale a două caractere vecine se numește distanță între pozițiile centrale.

Geometria plăcuței impune o constrângere apriori suplimentară asupra acestor distanțe: ultimele cinci caractere au aceeași distanță între pozițiile centrale, în timp ce distanța d_0 dintre al doilea și al treilea caracter este mai mare decât distanța d_1 dintre celelalte caractere ($d_0 > d_1$). Experimental, cele două mărimi se află în relația

$$d_0 = \lambda \cdot d_1, \quad \lambda \in [1.2, 1.3]. \quad (3.1)$$

Folosind aceste cunoștințe apriori, procesul de extragere a caracterelor parcurge următorii pași:

1. Din zona conexă pre-extrasă se determină intervalul vertical h_0 în care se află caracterele plăcuței.
2. Tot din zona conexă pre-extrasă se obțin lățimea caracterului w_1 și pozițiile centrale ale caracterelor, ținând cont de constrângerile geometrice de mai sus (lățime constantă și distanțele d_0, d_1 dintre pozițiile centrale).
3. Se scanează zonele conexe cuprinse în intervalul h_0 din imaginea plăcuței, integrând informația de poziție furnizată de proiecție; în funcție de rezultat se realizează fie combinarea mai multor zone într-un singur caracter, fie divizarea unei zone care înglobează mai multe caractere.
4. Se furnizează rezultatul extragerii, care conține pozițiile caracterelor, acestea putând fi astfel izolate cu precizie.

Acest proces adițional de selecție a regiunilor candidat sporește întreaga fiabilitate a sistemului deoarece elementele plăcuței care nu aparțin în-sine de numărul de înmatriculare precum șuruburile de fixare extrase în fig. 3.2 nu vor fi considerate mai departe drept zone de interes.

3.0.4 Analiza conturului caracterelor

O altă abordare are la bază detecția conturilor caracterelor. Pentru aceasta, în [12] este propus un proces compus din doi pași, fundamentat pe tehnici de segmentare ce utilizează metoda *Fast Marching* ghidată de formă (shape-driven).

Metoda *Fast Marching*, introdusă de Sethian [13], este un algoritm numeric folosit pentru a urmări evoluția unui front (a unei curbe) care se propagă monoton într-o singură direcție, dinspre o regiune inițială către exterior. Algoritmul este o particularizare a metodelor de tip *level-set*, în care viteza de propagare este strict pozitivă, astfel încât odată ce frontul a depășit un punct nu se mai poate întoarce.

Din punct de vedere matematic, metoda calculează, pentru fiecare punct x al imaginii, momentul de timp $T(x)$ la care frontul ajunge în acel punct. Această

mărime este soluția ecuației eikonale:

$$|\nabla T(x)| F(x) = 1, \quad (3.2)$$

unde $F(x)$ reprezintă funcția de viteză, care determină cât de rapid avansează frontul în fiecare punct. În aplicațiile de segmentare, funcția de viteză este derivată din informația din imagine (de regulă din magnitudinea gradientului): frontul se deplasează rapid în zonele omogene și încetinește, oprindu-se, în apropierea muchiilor, unde gradientul este mare. În acest fel, contururile obiectelor sunt delimitate în mod natural de zonele în care frontul stagnează.

Particularitatea abordării din [12] constă în ghidarea propagării frontului nu doar de informația de contur, ci și de modele statistice ale formei caracterelor (*shape-driven*). Funcția de viteză este astfel construită încât să integreze cunoștințe a priori despre forma caracterelor, permițând curbei să se oprească în dreptul muchiilor reale ale obiectelor și, în același timp, să inspecteze gradul de similaritate dintre conturul obținut și formele caracterelor cunoscute. Procesul realizează concomitent segmentarea și recunoașterea: pe parcursul evoluției frontului sunt determinate atât pozițiile delimitatoare ale fiecărui caracter, cât și eticheta (clasa) caracterului corespunzător, eliminând necesitatea unei etape separate de recunoaștere.

Capitolul 4

Recunoașterea caracterelor

Optofonul, dezvoltat în anul 1914, a fost unul dintre primele aparate electronice destinate citirii automate a textului tipărit. Funcționarea sa se baza pe proprietatea seleniului de a conduce curentul electric în mod diferit în funcție de intensitatea luminii: pe măsură ce dispozitivul scana o pagină, distingea zonele întunecate, corespunzătoare textului, de spațiile albe dintre cuvinte.

De la optofon și până astăzi, domeniul recunoașterii automate a caracterelor (OCR) a cunoscut progrese remarcabile, evoluând de la simpla detecție a contrastului către modele capabile să interpreteze caractere în condiții reale, cu variații de font, dimensiune și iluminare. În cele ce urmează sunt prezentate fundamentele teoretice ale celor mai importante abordări, de la metoda clasică a potrivirii de șabloane până la clasificatorii statistici, cu accent pe aplicarea acestora în recunoașterea numerelor de înmatriculare.

4.1 Potrivire de șabloane

Această abordare, cunoscută sub denumirea de template matching, reprezintă metoda clasică de recunoaștere a caracterelor prin compararea directă a pixelilor. Procesul presupune existența unei baze de date cu șabloane (matrice) predefinite pentru fiecare caracter (litere și cifre), care sunt comparate succesiv cu regiunea segmentată până la identificarea celei mai bune potriviri.

Deși este eficientă din punct de vedere computațional datorită simplității algoritmice, această tehnică prezintă limitări severe în scenarii reale. Ea este aplicabilă cu succes doar atunci când caracterele extrase au dimensiuni, fonturi și grosimi constante, fiind extrem de sensibilă la variații geometrice.

Pentru a introduce un grad de robustețe și pentru a atenua sensibilitatea la zgomot sau mici distorsiuni, în locul unei suprapuneri binare perfecte, se utilizează al-

goritmi bazați pe calcularea distanțelor matematice între vectorii de trăsături: distanța Hamming [14], valoarea Jaccard [15].

În mod concret, [14] prezintă un sistem de recunoaștere a caracterelor format din două etape: normalizarea caracterelor și potrivirea șabloanelor. Prima etapă presupune aducerea regiunilor de interes extrase în etapa anterioară la o dimensiune standard de 40×40 prin intermediul unei transformări afine care mapează pixelii din imaginea originală la cei ai regiunii determinate anterior.

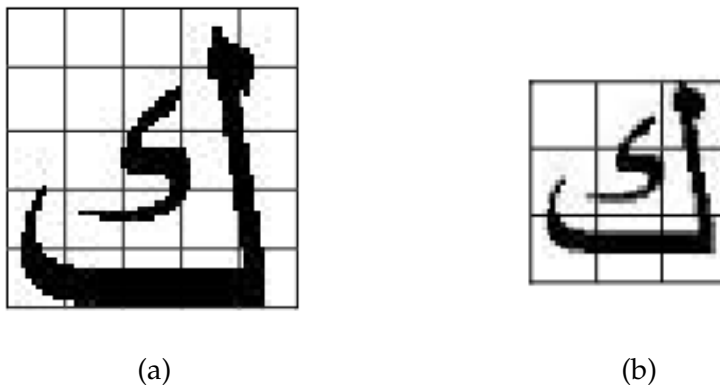


Figura 4.1: (a) Caracterul original, (b) Caracterul normalizat la o dimensiune standard de 40×40 [14].

Pentru etapa de potrivire a șabloanelor, precum am menționat anterior, autorii se folosesc de distanța Hamming pentru a determina cel mai apropiat caracter. Formula acestei distanțe este descrisă de Ecuația 4.1.

$$\sum_{i=1}^{nrows} \sum_{j=1}^{ncols} \text{mismatch}_{i,j} \quad (4.1)$$

$$\text{mismatch}_{i,j} = \begin{cases} 1, & \text{if } \text{original}_{i,j} \neq \text{extracted}_{i,j} \\ 0, & \text{if } \text{original}_{i,j} = \text{extracted}_{i,j} \end{cases}$$

Unde $nrows$ și $ncols$ sunt numărul de linii respectiv cel de coloane.

4.2 Clasificatori statistici

Această metodă valorifică avansurile recente din domeniul inteligenței artificiale pentru a crea module de recunoaștere a caracterelor (OCR) robuste. Acestea prezintă o rezistență ridicată la variațiile de luminozitate, dimensiuni sau fonturi variate utilizate în crearea numerelor de înmatriculare, etc. În literatura de specialitate, cele mai frecvent utilizate modele pentru această etapă a procesării imaginii sunt rețelele ne-

urionale artificiale (ANN) și mașinile pe suport vectorial (SVM), ambele prezentând avantaje și limitări specifice.

S-ar putea spune că există o oarecare similitudine între SVM și ANN deoarece ambele își au rădăcinile în algoritmul perceptronului [16], însă abordările matematice care succedă acest algoritm diferă fundamental.

4.2.1 Mașini pe suport vectorial

Algoritmul ce stă la baza mașinilor pe suport vectorial reprezintă o generalizare a algoritmului "portret generalizat" dezvoltat la începutul anilor 1960 [17], [18]. Fundamentele matematice ale regresiei/ clasificării vectoriale sunt ancorate în învățarea statistică și teoria VC, care oferă o măsură a capacității de învățare a unui algoritm de învățare automată [19].

Este important de menționat că algoritmul perceptronului funcționează prin actualizarea vectorului de greutate $\mathbf{w}$ pornind de la o valoare inițială arbitrară, de regulă vectorul nul. Acesta este modificat prin adunarea exemplilor pozitive clasificate în mod eronat ca fiind negative, sau prin scăderea exemplilor negative clasificate ca fiind pozitive. Așadar, rezultatul final va fi întotdeauna o combinație liniară a punctelor din setul de antrenament.

$$\mathbf{w} = \sum_{i=1}^{\ell} \alpha_i y_i \mathbf{x}_i,$$

Unde semnul ecuației este dat de clasificarea $y_i \in \{-1, 1\}$, iar valorile α_i sunt valori pozitive proporționale cu numărul de misclasificări făcute asupra punctului în timpul stagiului de antrenament. Punctele care au cauzat mai puține erori, vor avea un α_i mai mic, iar punctele dificile unul mai mare. Date fiind aceste valori, perceptronul poate fi rescris folosind următoarea formulă:

$$h(\mathbf{x}) = \text{sgn} \left(\sum_{j=1}^{\ell} \alpha_j y_j \langle \mathbf{x}_j, \mathbf{x} \rangle + b \right)$$

numită și forma duală a perceptronului.

Învățarea în spațiul trăsăturilor

Complexitatea funcției care trebuie aproximată de modelul de învățare depinde în mod direct de modul în care sunt reprezentate datele de intrare; astfel, dificultatea procesului de învățare poate varia semnificativ în funcție de această reprezentare. În mod ideal, se urmărește alegerea unei reprezentări adaptate specificului problemei

de învățare. O metodă frecvent utilizată în acest scop o constituie transformarea reprezentării datelor printr-o funcție de mapare a trăsăturilor, care proiectează fiecare punct din spațiul de intrare într-un nou spațiu, denumit spațiul trăsăturilor:

$$x = (x_1, x_2, \dots, x_n) \mapsto \phi(x) = (\phi_1(x), \phi_2(x), \dots, \phi_N(x)).$$

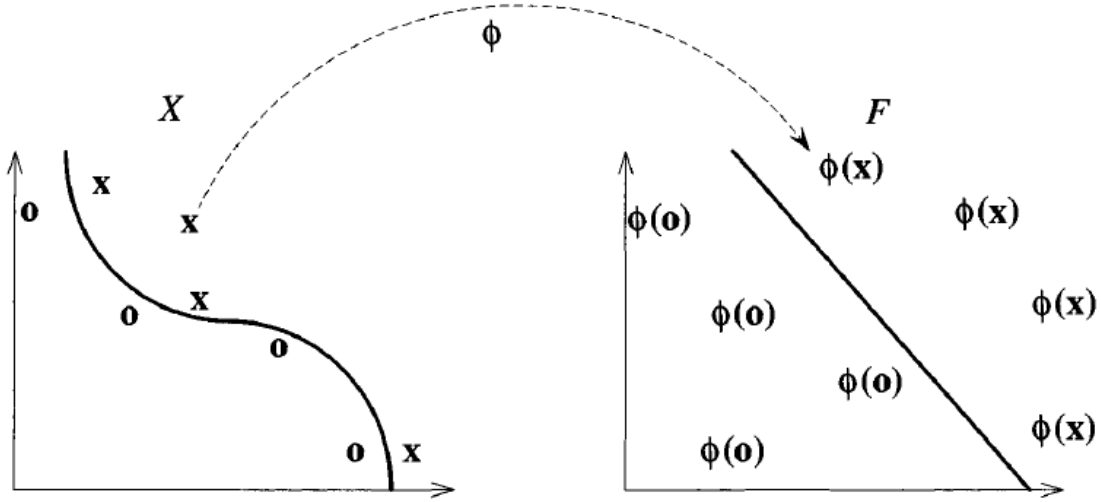


Figura 4.2: O funcție de mapare a trăsăturilor poate simplifica problema de clasificare, transformând un set nelinier separabil într-unul liniar separabil [20].

Datorită varietății mari a problemelor de învățare, spațiul trăsăturilor rezultat are, de regulă, o dimensiune mult mai mare decât spațiul de intrare (uneori chiar infinită). În consecință, maparea explicită a fiecărui punct în acest spațiu și rezolvarea directă a problemei de învățare, de forma

$$f(x) = \sum_{i=1}^N w_i \phi_i(x) + b,$$

devin nefezabile din punct de vedere computațional, întrucât ar necesita calcularea și stocarea explicită a tuturor celor N componente $\phi_i(x)$.

Reprezentarea duală introdusă anterior permite însă rescrierea problemei de învățare în spațiul trăsăturilor sub forma:

$$f(x) = \sum_{i=1}^{\ell} \alpha_i y_i \langle \phi(x_i), \phi(x) \rangle + b.$$

În această formulare, datele de intrare apar exclusiv prin intermediul produsului scalar $\langle \phi(x_i), \phi(x) \rangle$ din spațiul trăsăturilor. Prin urmare, dacă acest produs scalar ar putea fi calculat direct, ca o funcție definită pe punctele din spațiul de intrare,

fără a construi explicit maparea ϕ , atunci cei doi pași (maparea datelor și rezolvarea problemei de învățare) ar putea fi combinați, obținând mașini de învățare capabile să separe date care nu sunt liniar separabile în spațiul original.

O astfel de funcție de mapare implicită din spațiul de intrare în cel al trăsăturilor se numește funcție nucleu (*kernel*) și stă la baza dezvoltării mașinilor pe suport vectorial, pas numit în literatura de specialitate *kernel trick*.

Teoria VC și clasificatori cu marjă maximală

Maparea implicită în spațiul trăsăturilor, realizată prin intermediul funcțiilor nucleu, permite mașinilor pe suport vectorial să definească soluții în spații de dimensiune foarte mare, eventual chiar infinită. Această flexibilitate constituie însă o sabie cu două tăișuri: cu cât familia de funcții pe care modelul o poate reprezenta este mai bogată, cu atât crește și riscul ca acesta să se muleze perfect pe setul de antrenament memorându-l în întregime fără a surprinde structura reală a datelor și, prin urmare, fără a generaliza la exemple noi. Se ridică astfel problema fundamentală a cuantificării capacității unui model, cu scopul de a controla echilibrul dintre fidelitatea față de datele de antrenament și capacitatea de generalizare.

Această problemă este abordată de teoria Vapnik-Chervonenkis (VC), dezvoltată în strânsă legătură cu cadrul învățării *probabil aproximativ corecte* (*Probably Approximately Correct*, PAC) [21], care formalizează tocmai noțiunea de învățare cu garanții asupra generalizării. Tocmai pentru că oferă instrumentele necesare controlului capacității unui model, teoria VC reprezintă fundamentul teoretic indispensabil al mașinilor pe suport vectorial.

Conceptul central al acestei teorii îl constituie dimensiunea VC, o măsură a capacității de exprimare a unei familii de funcții de clasificare. În mod formal, dimensiunea VC reprezintă cardinalitatea celui mai mare set de puncte care poate fi etichetat (pulverizat) în toate modurile binare posibile de către funcțiile clasei respective. O dimensiune VC ridicată indică un model cu o capacitate sporită de a se mula pe orice configurație a datelor, dar și un risc proporțional crescut de supraînvățare, deoarece marja garantată între eroarea empirică (pe setul de antrenament) și eroarea reală (așteptată pe distribuția datelor) crește odată cu dimensiunea VC. În consecință, controlul acestei dimensiuni reprezintă o cerință esențială pentru asigurarea capacității de generalizare a modelului.

Pe baza acestei mărimi, teoria VC stabilește o legătură cantitativă între eroarea măsurată pe setul de antrenament și eroarea așteptată pe întreaga distribuție a datelor. Astfel, pentru orice ipoteză h aparținând clasei de funcții analizate, capacitatea de generalizare poate fi mărginită superior, cu o probabilitate de cel puțin $1 - \delta$, prin

următoarea inegalitate:

$$err_D(h) \leq \epsilon(l, H, \delta) = \frac{2k}{l} + \frac{4}{l} \left(d \log \frac{2el}{d} + \log \frac{4}{\delta} \right), \quad (4.2)$$

unde semnificația fiecărui termen este următoarea:

- $err_D(h)$ — eroarea reală (de generalizare) a ipotezei h , definită ca probabilitatea ca aceasta să clasifice greșit un exemplu extras aleator din distribuția D a datelor;
- $\epsilon(l, H, \delta)$ — marginea superioară garantată asupra erorii de generalizare, exprimată în funcție de dimensiunea setului de antrenament, de complexitatea clasei de funcții și de nivelul de încredere impus;
- l — numărul de exemple din setul de antrenament; pe măsură ce l crește, marginea descrește, reflectând faptul că un volum mai mare de date conduce la garanții mai stricte asupra generalizării;
- H — clasa de funcții de clasificare (ipoteze) din care este selectată soluția;
- d — dimensiunea VC a clasei H , care cuantifică capacitatea de exprimare a acesteia; o valoare mare a lui d mărește marginea, semnalând un risc sporit de supra-învățare;
- k — numărul de exemple clasificate eronat pe setul de antrenament (eroarea empirică), care exprimă fidelitatea ipotezei față de datele observate;
- δ — nivelul de încredere: inegalitatea este satisfăcută cu o probabilitate de cel puțin $1 - \delta$, astfel încât δ reprezintă probabilitatea (mică) ca marginea să nu fie respectată;
- e — baza logaritmului natural (constanta lui Euler).

Structura acestei margini evidențiază compromisul fundamental dintre fidelitatea față de datele de antrenament și capacitatea de generalizare: primul termen, $2k/l$, penalizează erorile empirice, în timp ce al doilea termen crește odată cu dimensiunea VC d , penalizând astfel complexitatea excesivă a clasei de funcții. Minimizarea simultană a celor doi termeni constituie esența principiului minimizării riscului structural, pe care se fundamentează mașinile pe suport vectorial.

În mod concret, mașinile pe suport vectorial restrâng familia clasificatorilor și dimensiunea VC a acestora prin adăugarea următoarei constrângeri: hiperplanul care separă elementele din setul de antrenament trebuie să fie unul cu marja maximală. Adăugând această constrângere și presupunând că punctele de antrenament

sunt incluse într-o sferă de rază R , atunci dimensiunea VC efectivă a clasei rezultate este mărginită superior de raportul R^2/γ^2 [20], unde γ reprezintă marja geometrică a hiperplanului separator.

Formal, problema clasificatorului cu marjă maximală se reduce la rezolvarea problemei de optimizare convexă:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{cu restricția} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, \ell,$$

unde minimizarea normei $\|\mathbf{w}\|$ este echivalentă cu maximizarea marjei $\gamma = 1/\|\mathbf{w}\|$. Această proprietate este deosebit de relevantă în contextul recunoașterii caracterelor de pe plăcuțele de înmatriculare, deoarece spațiile de trăsături utilizate (vectori obținuți prin caracteristici HOG, momente Zernike sau pixeli normalizați) sunt frecvent înalt dimensionale, iar capacitatea unui clasificator de a evita supra-învățarea într-un astfel de regim este critică.

Pentru a permite tratarea seturilor de date care nu sunt liniar separabile, formularea originală a fost extinsă prin introducerea variabilelor de relaxare ξ_i și a parametrului de penalizare C [22], conducând la formularea cu marjă moale:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^{\ell} \xi_i \quad \text{cu restricția} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

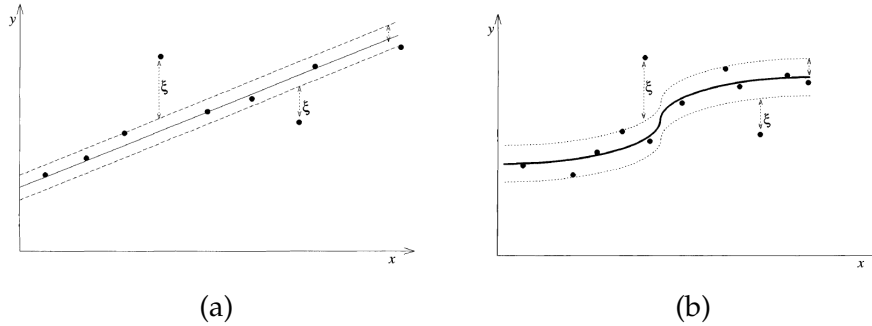


Figura 4.3: (a) Variabila de relaxare pentru exemple liniar separabile, (b) Variabila de relaxare pentru exemple non-liniar separabile

Parametrul C controlează raportul dintre maximizarea marjei și tolerarea erorilor de clasificare, permițând astfel un compromis explicit între capacitatea modelului și riscul empiric. Combinat cu utilizarea funcțiilor kernel introduse prin [23], acest cadru permite construirea unor clasificatori cu putere de discriminare ridicată, păstrând în același timp un control formal asupra capacității de generalizare.

4.2.2 Rețele neuronale artificiale

Rețelele neuronale artificiale (ANN) reprezintă cea de-a doua direcție majoră în recunoașterea caracterelor din cadrul plăcuțelor de înmatriculare. Spre deosebire de SVM, care își are fundamentul în teoria învățării statistice, ANN-urile sunt inspirate dintr-un model simplificat al activității neuronilor biologici.

Această abordare inspirată din anatomia umană nu poate părea evidentă în mod inițial, însă de-a lungul istoriei evoluția a conferit creierului uman o multitudine de caracteristici de dorit care nu sunt prezente într-o mașină von Neumann sau o mașină de calcul modernă:

- Paralelism uriaș
- Capacitatea de învățare
- Capacitatea de generalizare
- Adaptabilitate

și se bazează pe compunerea ierarhică a unităților elementare derivate din perceptronul lui Rosenblatt [16].

Arhitectura perceptronului multistrat

Din punct de vedere structural, o rețea neuronală artificială poate fi modelată ca un graf orientat și ponderat, în care nodurile corespund neuronilor artificiali, iar muchiile descriu conexiunile prin care ieșirea unui neuron devine intrare pentru un altul. Fiecărei conexiuni îi este asociată o pondere ce reglează intensitatea semnalului transmis, ponderi care sunt ajustate pe parcursul procesului de învățare.

Perceptronul multistrat (MLP) este alcătuit dintr-un strat de intrare, unul sau mai multe straturi ascunse și un strat de ieșire; adoptând convenția uzuală conform căreia nodurile de intrare nu sunt considerate un strat propriu-zis, o rețea cu L straturi cuprinde $L - 1$ straturi ascunse și un strat de ieșire. Fiecare neuron j dintr-un strat efectuează o transformare afină asupra activărilor stratului precedent, urmată de aplicarea unei funcții de activare neliniare σ :

$$a_j = \sigma \left(\sum_i w_{ji} x_i + b_j \right).$$

O interpretare geometrică intuitivă a acestei arhitecturi evidențiază rolul fiecărui strat în construirea frontierei de decizie: fiecare unitate din primul strat ascuns definește un hiperplan care separă spațiul de intrare, neuronii celui de-al doilea strat combină aceste hiperplane în regiuni de decizie, iar stratul de ieșire le reunește,

printr-o operație de tip conjuncție (AND) a separărilor obținute, într-o regiune de decizie finală de formă arbitrară.

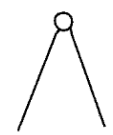
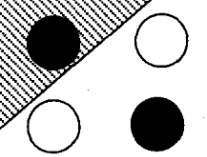
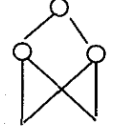
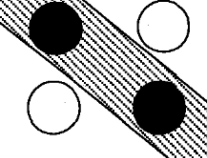
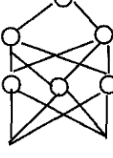
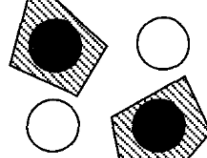
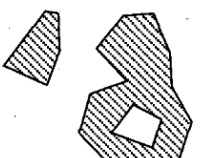
Structure	Description of decision regions	Exclusive-OR problem	Classes with meshed regions	General region shapes
 Single layer	Half plane bounded by hyperplane			
 Two layer	Arbitrary (complexity limited by number of hidden units)			
 Three layer	Arbitrary (complexity limited by number of hidden units)			

Figura 4.4: O interpretare geometrică a rolului straturile ascunse [24]

Compoziția acestor transformări permite rețelei să construiască, în mod implicit, reprezentări ale datelor de complexitate crescândă. Justificarea teoretică pentru această abordare este oferită de teorema aproximării universale, care stabilește că o rețea cu un singur strat ascuns și un număr suficient de neuroni poate aproxima, cu precizie arbitrară, orice funcție continuă definită pe un domeniu compact [25].

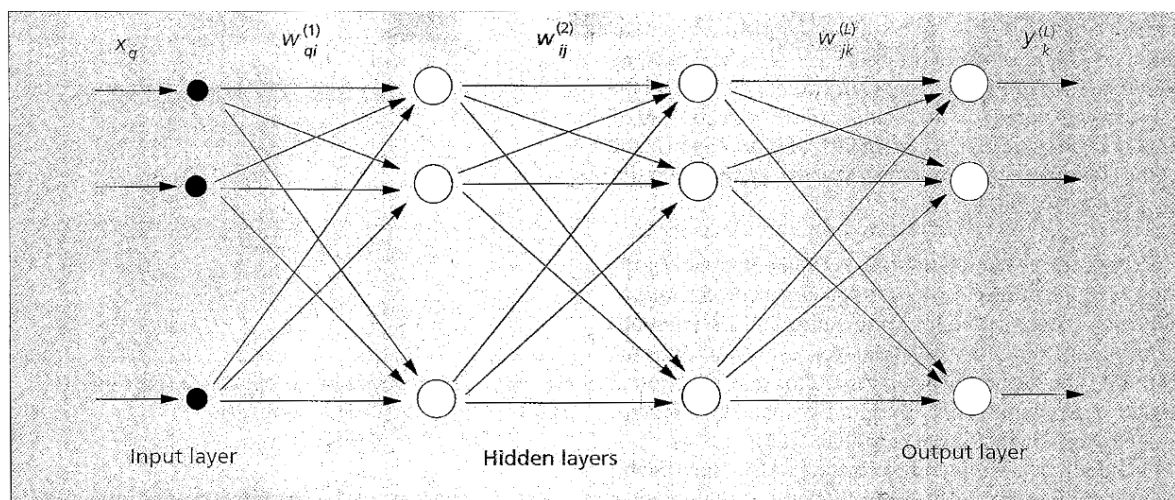


Figura 4.5: Arhitectura unei rețele de tip MLP cu 3 straturi [24]

Antrenarea prin retropropagarea erorii

Spre deosebire de mașinile pe suport vectorial, a căror antrenare se reduce la o problemă de optimizare convexă cu soluție unică, antrenarea unei rețele neuronale presupune minimizarea unei funcții de eroare puternic neliniare în raport cu ponderile, ceea ce conduce, în general, la o suprafață de eroare cu numeroase minime locale.

Procesul de învățare pornește de la definirea unei funcții de cost (sau de eroare) care cuantifică discrepanța dintre ieșirile produse de rețea și valorile țintă asociate exemplurilor de antrenament. O alegere uzuală o constituie eroarea pătratică:

$$E = \frac{1}{2} \sum_k (t_k - y_k)^2,$$

unde t_k reprezintă valoarea dorită, iar y_k ieșirea efectivă a neuronului k din stratul de ieșire. Antrenarea revine astfel la ajustarea iterativă a ponderilor în direcția care reduce această eroare, folosind metoda coborârii gradientului: fiecare pondere este modificată proporțional cu gradientul negativ al erorii în raport cu ea,

$$w_{ji} \leftarrow w_{ji} - \eta \frac{\partial E}{\partial w_{ji}},$$

unde η este rata de învățare, un hiperparametru care controlează amplitudinea pasului de actualizare.

Dificultatea principală constă în calcularea derivatelor parțiale $\partial E / \partial w_{ji}$ pentru ponderile situate în straturile ascunse, unde contribuția la eroarea finală nu este una directă. Algoritmul de retropropagare a erorii [26] rezolvă această problemă aplicând în mod sistematic regula de derivare în lanț, distribuind eroarea de la ieșire înapoi către straturile anterioare. El se desfășoară în două etape:

- o etapă de propagare înainte (*forward pass*), în care exemplul de intrare traversează rețeaua strat cu strat, producând activările tuturor neuronilor și, în final, ieșirea rețelei;
- o etapă de propagare înapoi (*backward pass*), în care eroarea calculată la nivelul stratului de ieșire este propagată în sens invers, dinspre ieșire spre intrare, atribuind fiecărui neuron o cotă de responsabilitate pentru eroarea totală și permițând astfel calcularea gradientului pentru fiecare pondere.

Cele două etape se repetă pentru exemplele setului de antrenament până la convergența erorii către un minim. Deoarece suprafața de eroare nu este convexă, algoritmul nu garantează atingerea minimului global, putând rămâne blocat într-un minim local; în practică, însă, soluțiile obținute oferă o capacitate de generalizare satisfăcătoare.

Eficiența și stabilitatea procesului depind în mod esențial de alegerea ratei de învățare și de strategia de actualizare a ponderilor, care poate fi aplicată după fiecare exemplu, pe loturi reduse de exemple (*mini-batch*) sau asupra întregului set de antrenament.

Paradigme de utilizare în recunoașterea caracterelor

Figura 4.6 prezintă principalele două paradigme de utilizare a rețelelor neuronale în cadrul sistemelor de recunoaștere a caracterelor. Prima dintre ele implică un extractor de trăsături explicit, care transmite mai departe rezultatele sale la stratul de intrare al rețelei neuronale. Paradigma alternativă nu extrage în mod explicit trăsăturile caracterelor, acest lucru se întâmplă în mod implicit în cadrul straturilor ascunse. O proprietate avantajoasă a acestei paradigme este faptul că extragerea trăsăturilor și clasificarea sunt antrenate în mod simultan pentru a produce rezultate optime.

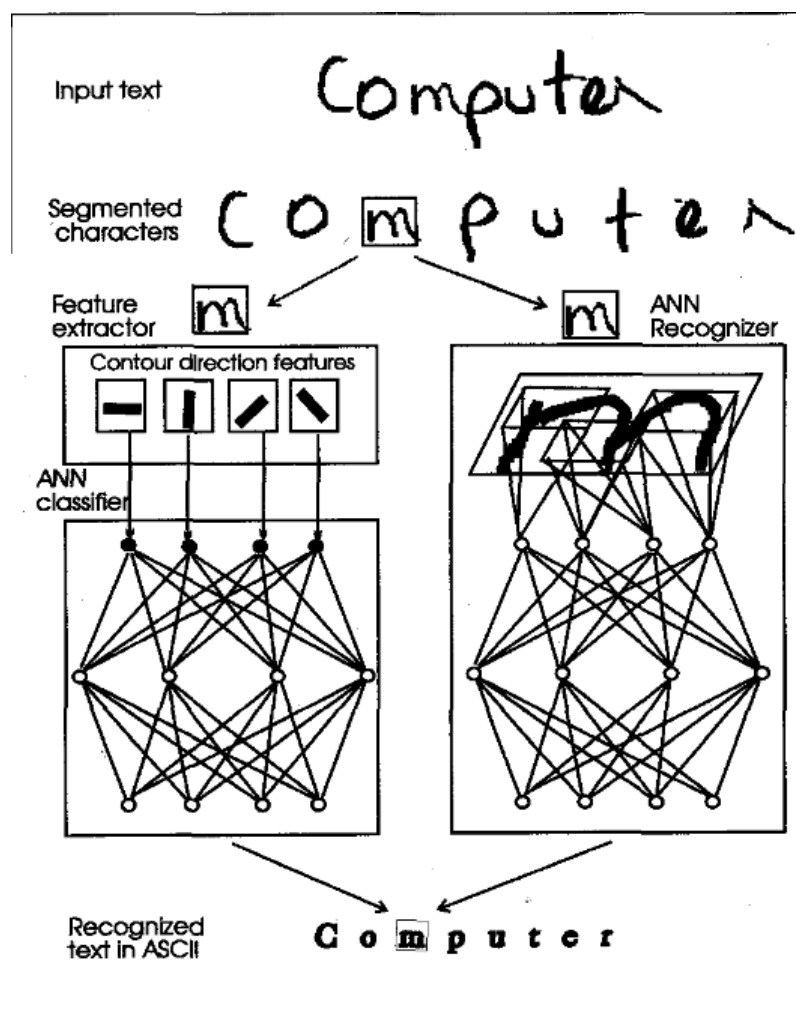


Figura 4.6: Două variante pentru folosirea ANN-urilor în contextul OCR-ului [24]

Capitolul 5

Implementarea sistemului IORA

Sistemul propus în lucrarea de față, denumit IORA, a fost construit pornind de la o arhitectură modulară formată din trei componente cheie: modulul de preluare a datelor (*Input Module*), fluxul de procesare (*Processing Pipeline*) și modulul de logică a aplicației (*Application Logic Module*). Pentru a oferi flexibilitate și a respecta principiile de proiectare extensibilă specifice conceptului de clean code [27], modulul de intrare și cel de logică a aplicației utilizează polimorfismul prin expunerea unor interfețe abstracte.

Această abordare decuplată permite diferitelor aplicații să integreze nucleul sistemului într-un mod elegant, implementând comportamente personalizate și adaptate la mediul de execuție, fără a modifica codul de bază. De exemplu, modulul de intrare poate fi extins pentru a prelua fluxuri video de la o cameră atașată contextului de execuție, fie prin intermediul socket-urilor. În mod similar, modulul de logică a aplicației poate fi personalizat pentru a declanșa acțiuni specifice precum trimiterea unor notificări automate prin e-mail, sau acționarea unor echipamente *hardware* (ridicarea unei bariere auto), în cazul unui sistem embedded.

În contrast, fluxul de procesare este încapsulat ca o unitate de execuție independentă și de sine stătătoare, neexpunând nicio interfață publică destinată extinderii structurale.

În continuare, acest capitol descrie arhitectura și tehnologiile utilizate în implementarea aplicației prezentate alături de o descriere tehnică a fiecărui modul.

5.1 Arhitectura sistemului

Sistemul IORA este organizat pe trei niveluri logice care corespund celor trei module introduse la începutul capitolului: nivelul de achiziție a datelor, nivelul de procesare și nivelul de logică a aplicației. Această stratificare urmărește o separare clară a responsabilităților, fiecare nivel comunicând cu cel adiacent exclusiv prin

structuri de date simple, independente de bibliotecile externe folosite în implementare. Decuplarea astfel obținută permite ca un nivel să fie înlocuit sau extins fără a afecta restul sistemului.

5.1.1 Diagrama de componente

Diagrama de componente surprinde cele trei module și fluxul de date dintre ele. Modulul de intrare expune interfața `ICaptureManager` și produce, indiferent de sursa fizică a datelor, obiecte de tip `Frame`. Aceste obiecte constituie singurul punct de contact dintre modulul de intrare și fluxul de procesare, eliminând orice dependență a nucleului de natura sursei de captură.

Fluxul de procesare consumă obiectul `Frame` și aplică, în mod secvențial, cele trei etape ale procesului de recunoaștere: localizarea plăcuței, segmentarea caracterelor și recunoașterea acestora. Rezultatul produs (șirul de caractere recunoscut, însoțit de metadatele asociate) este transmis mai departe către modulul de logică a aplicației, care decide acțiunea declanșată în funcție de contextul de execuție.

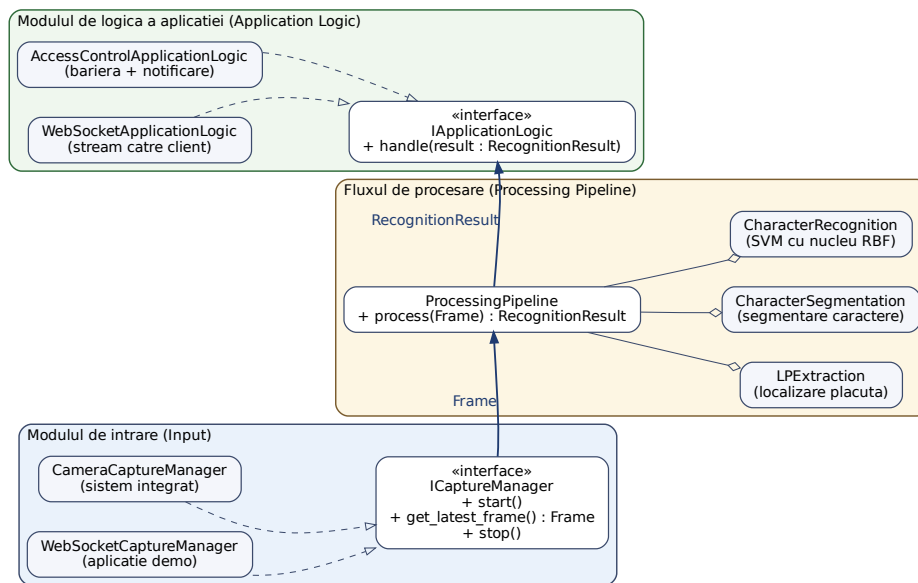


Figura 5.1: Diagrama de componente a sistemului IORA: modulele de intrare, procesare și logică a aplicației, împreună cu structurile de date `Frame` și `RecognitionResult` care le decuplează.

5.1.2 Actori și cazuri de utilizare

În cadrul scenariului de utilizare vizat, sistemul interacționează cu mai mulți actori, atât umani, cât și automați. Actorul uman principal este *operatorul*, responsabil

de configurarea și monitorizarea sistemului. Actorii automați sunt reprezentați de echipamentele și serviciile care consumă rezultatul recunoașterii: *sistemul de barieră* (acționat în urma unei recunoașteri valide) și *serviciul de notificare* (care transmite alerte, de exemplu prin e-mail).

Principalele cazuri de utilizare sunt: monitorizarea fluxului video de la sursa de captură, citirea automată a numărului de înmatriculare dintr-un cadru și declanșarea unei acțiuni asociate (ridicarea barierei sau emiterea unei notificări) pe baza rezultatului recunoașterii. Pentru aplicația demonstrativă, se adaugă cazul de utilizare al transmiterii rezultatelor către un client extern prin intermediul unei conexiuni WebSocket.

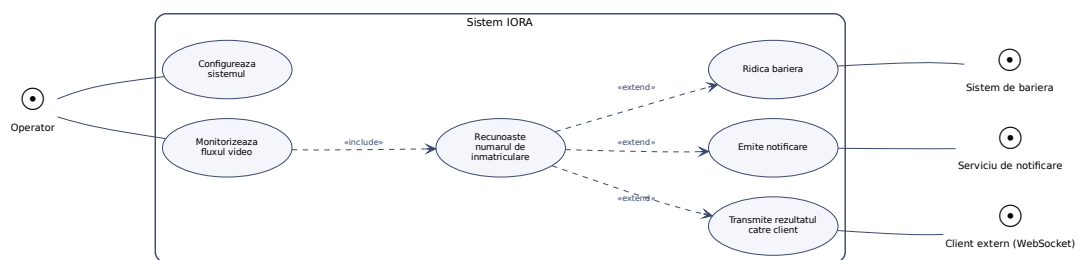


Figura 5.2: Diagrama cazurilor de utilizare a sistemului IORA, cu operatorul drept actor uman principal și sistemele consumatoare ale rezultatului recunoașterii drept actori automați.

5.1.3 Diagrama de clase

Diagrama de clase din figura 5.3 reflectă, la nivel structural, aceeași stratificare introdusă de diagrama de componente: interfețele `ICaptureManager` și `IApplicationLogic` delimitează marginile extensibile ale sistemului, fiecare având două realizări concrete (una pentru sistemul integrat, una pentru aplicația demonstrativă), în timp ce nucleul de procesare este modelat prin clasa `ProcessingPipeline`, care agregă cele trei etape ale recunoașterii (`LPExtraction`, `CharacterSegmentation`, `CharacterRecognition`). Tipurile de date proprii `Frame` și `RecognitionResult` apar ca dependențe la marginile fluxului, marcând explicit decuplarea nucleului de detaliile de implementare ale surselor de captură și ale acțiunilor declanșate de aplicație.

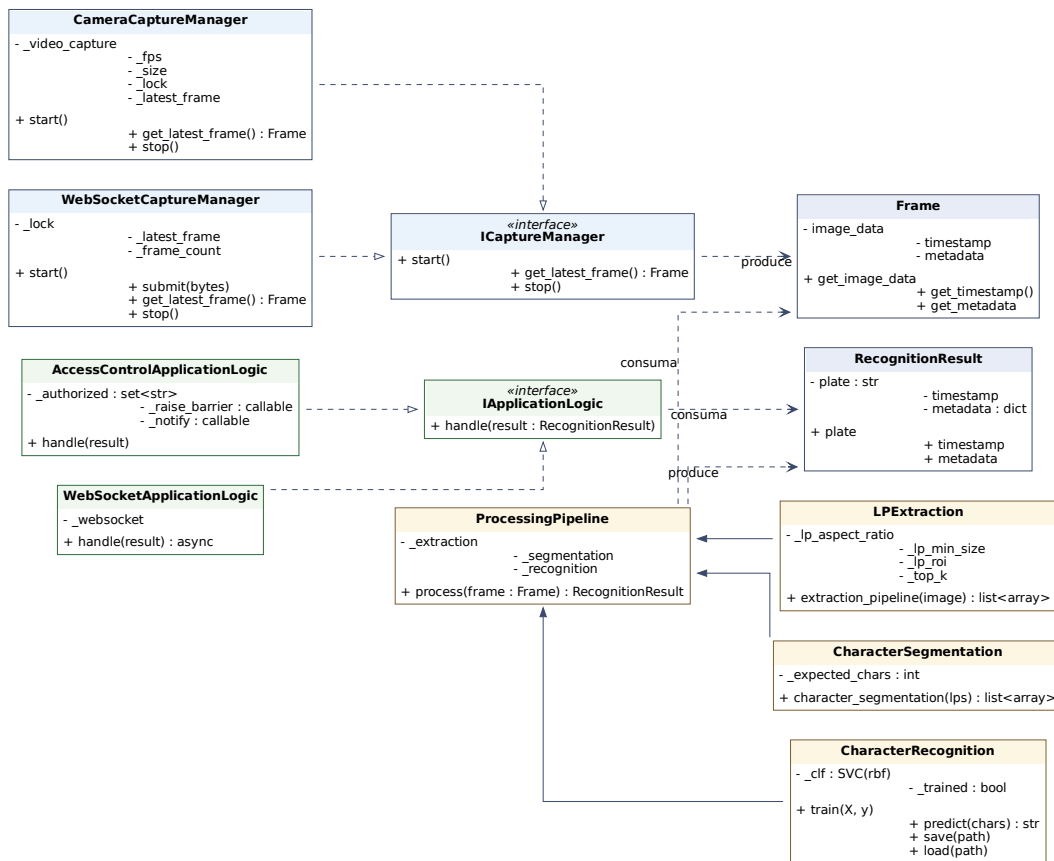


Figura 5.3: Diagrama de clase a sistemului IORA: interfețele extensibile, realizările lor concrete și agregarea celor trei etape de procesare în **ProcessingPipeline**.

5.1.4 Decizii de proiectare

Decizia arhitecturală fundamentală a sistemului constă în expunerea unor interfețe abstracte la marginile fluxului de date (modulul de intrare și modulul de logică a aplicației), în timp ce fluxul de procesare este încapsulat ca o unitate închisă, fără puncte de extindere structurală. Această asimetrie reflectă natura componentelor: modulele de la margini interacționează direct cu mediul de execuție (componente hardware, sistem de operare, servicii externe) și sunt, prin urmare, cele mai susceptibile la variații, motiv pentru care comportamentul lor este abstractizat prin interfețe conform principiilor de *clean code* [27]. În schimb, fluxul de procesare implementează un algoritm stabil, independent de context, care nu beneficiază de pe urma extensibilității.

O a doua decizie de proiectare vizează minimizarea dependențelor la nivelul nucleului și definirea unor tipuri de date proprii (precum obiectul **Frame**), independente de orice bibliotecă externă. Această alegere consolidează portabilitatea sistemului și permite adoptarea sa în contexte de rulare diferite, de la sisteme inte-

grate cu resurse limitate până la servere care deserveșc o aplicație web.

5.2 Tehnologii

Alegerea limbajului a fost făcută pe considerente practice, așadar întreg sistemul a fos implementat în Python 3.14.4 în detrimenul unui limbaj compilat precum C++. Această decizie a fost motivată de gradul ridicat de adopție a ecosistemului Python în cadrul sistemelor integrate, precum și de flexibilitatea și viteza de dezvoltare specifice oferite de un limbaj dinamic. De asemenea, librăriile necesare implementării modului de procesare se bazează pe nuclee scrise în C, optimizate la nivel de instrucțiune.

Pentru a oferi o structură clară, tehnologiile utilizate au fost împărțite în două categorii: cele fundamentale, necesare dezvoltării nucleului sistemului (interfețele menționate anterior și fluxul de procesare), și cele adiționale, specifice dezvoltării aplicației finale.

5.2.1 Nucleu

În etapa de proiectare și dezvoltare a nucleului s-a urmărit minimizarea numărului de dependențe externe. Acest principiu sprijină portabilitatea, permițând adoptarea sistemului în diferite contexte de rulare.

Așadar, tehnologiile de bază sunt:

- OpenCV 4.13.0
- numpy 2.4.2
- scikit-learn 1.8.0
- matplotlib-base 3.10.9

5.2.2 Nivelul de aplicație

După cum am menționat, dependențele necesare dezvoltării aplicației finale sunt opționale, acestea depinzând strict de contextul aplicației dorite. Pentru aplicația propusă, dependențele sunt:

- fastapi 0.135.4
- websockets 15.0.1

5.3 Detalii tehnice

În cele ce urmează va fi prezentată o descriere detaliată a arhitecturii fiecărui modul, abordând aspecte precum interfețele de programare expuse, deciziile de proiectare și implementarea propriu-zisă.

Pentru modulele extensibile, se va realiza o paralelă între implementarea în cadrul unui sistem integrat și implementarea concretă realizată în cadrul aplicației demonstrate.

5.3.1 Modulul de intrare (Input)

Extensibilitatea și portabilitatea au constituit principalii factori care au ghidat proiectarea acestui modul. Decizia arhitecturală se fundamentează pe principiile de *clean code* [27], conform cărora secțiunile susceptibile la modificări; în special cele care interacționează direct cu componentele hardware sau cu nivelurile inferioare ale sistemului de operare; ar trebui izolate în clase dedicate, al căror comportament să fie abstractizat prin intermediul unei interfețe.

Interfața `ICaptureManager`

Interfața `ICaptureManager` este definită prin trei metode esențiale. Primele două, `start` și `stop`, gestionează ciclul de viață al sistemului de captură, ocupându-se de inițializarea și, respectiv, de eliberarea resurselor asociate acestuia. Cea de-a treia metodă, `get_latest_frame`, returnează cel mai recent cadru preluat, încapsulat într-un obiect de tip `Frame`.

```
def start(self) -> None:
    pass

def get_latest_frame(self) -> Frame:
    pass

def stop(self) -> None:
    pass
```

Valoarea returnată este reprezentată prin obiectul `Frame`, o structură de date minimală care încapsulează cadrul propriu-zis (`image_data`), momentul temporal al capturii (`timestamp`) și un set opțional de metadate (`metadata`). Definirea unui tip de date propriu, independent de orice bibliotecă externă, menține interfața decuplată de detaliile de implementare ale unei surse de captură anume și consolidează portabilitatea nucleului.

Sistem integrat

Pentru scenariul rulării pe un sistem integrat, a fost realizată o implementare exemplificativă, clasa `CameraCaptureManager`, responsabilă de preluarea cadrelor de la o cameră conectată direct la dispozitiv.

Implementarea se bazează pe biblioteca OpenCV pentru a media interacțiunea cu subsistemul de captură video al sistemului de operare. Identificarea sursei fizice se realizează prin intermediul indexului de dispozitiv asociat camerei, transmis ca parametru constructorului clasei.

Pe lângă indexul de dispozitiv, clasa expune următoarele attribute private:

- `_video_capture` – obiectul OpenCV prin care se realizează comunicarea cu camera;
- `_fps` – numărul de cadre pe secundă raportat de dispozitiv;
- `_size` – rezoluția cadrelor preluate, dedusă din primele cadre citite;
- `_lock` – obiect necesar pentru sincronizarea accesului la cel mai recent cadru.

Metoda `get_latest_frame` a fost proiectată astfel încât să returneze întotdeauna cel mai recent cadru disponibil. Această cerință este esențială într-un context de procesare în timp real: întrucât bibliotecile de captură mențin o memorie internă, citirea secvențială a cadrelor ar introduce o latență cumulativă, returnând cadre deja învechite. Pentru a evita acest fenomen, un fir de execuție (*thread*) dedicat preia continuu cadrele de la cameră, păstrând disponibil doar cel mai recent dintre acestea. Accesul concurent la resursa partajată este sincronizat printr-un mecanism de tip *lock*.

```
def get_latest_frame(self) -> Frame:
    with self._lock:
        frame = self._latest_frame
        if frame is None:
            return None
        frame = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
        timestamp = self._video_capture.get(cv.CAP_PROP_POS_MSEC)
        metadata = {"source": "camera", "device_index": self._device_index}
        return Frame(frame, timestamp, metadata)
```

Aplicație demo

În cadrul aplicației demonstrative, sursa de captură nu este o cameră atașată direct dispozitivului, ci un flux video transmis în timp real (*live-feed*) printr-o conexiune WebSocket. Această implementare a interfeței `ICaptureManager` menține

o conexiune persistentă prin care cadrele sosesc continuu de la un client extern, le decodează și le încapsulează în același obiect `Frame`, păstrând astfel contractul interfeței neschimbat. La fel ca în cazul implementării pentru sistemul integrat, doar cel mai recent cadru recepționat este păstrat disponibil, evitând acumularea unei latențe în contextul procesării în timp real. Substituirea sursei fizice cu un flux primit prin rețea, fără nicio modificare a fluxului de procesare, demonstrează în practică beneficiul abstractizării prin interfață: nucleul rămâne complet indiferent la natura sursei de date.

5.3.2 Fluxul de procesare

Considerații preliminare

Precum am menționat când am definit contextul aplicației, există o diversitate ridicată în privința formatului, a paletelor cromatice de fond, a dimensiunilor relative și a fontului caracterelor. Tocmai din aceste motive, în prezenta lucrare a fost exclusă de la bun început orice abordare bazată pe segmentarea culorii (color-based segmentation), deși aceasta este frecvent utilizată în literatură pentru regiuni geografice cu plăcuțe standardizate cromatic. Lucrări reprezentative precum [2], care exploatează caracteristicile cromatice ale plăcuțelor taiwaneze, sau [3] pentru spațiul autohton al insulei Taiwan, demonstrează eficacitatea acestor metode atunci când mediul de operare prezintă o uniformitate cromatică ridicată. Generalizarea acestor abordări către scenariul nostru ar fi necesitat menținerea unui catalog de palete pentru fiecare format admis, ceea ce ar fi introdus o sursă suplimentară de erori și ar fi diminuat capacitatea de adaptare a sistemului la plăcuțele atipice sau la formatele nou introduse de către autorități.

Localizarea plăcuței de înmatriculare

În capitolul teoretic dedicat acestui sub-sistem au fost menționate trei abordări: bazate pe culoare, pe contururi sau pe textură. După cum am menționat anterior, cele bazate pe culoare nu sunt o variantă fiabilă deoarece pe teritoriul României există 4 combinații cromatice posibile care definesc un număr de înmatriculare valid.

Pe de altă parte, metodele bazate pe textură prezintă un grad al robusteții mai scăzut față de cele bazate pe contururi. Deși ambele categorii pornesc de la aceeași observație, anume că zona caracterelor produce o concentrare de tranziții de intensitate, ele diferă fundamental prin criteriul de decizie folosit pentru a confirma o regiune drept plăcuță. Abordările textuale ridică descriptorul texturii la rangul

de discriminator principal, fie sub forma unei hărți de densitate ponderată a liniilor verticale, fie prin măsurarea variației statistice locale a intensităților într-o vecinătate glisantă. Acest descriptor este însă parametrizat în raport cu textura așteptată a caracterelor: dimensiunea ferestrelor de analiză, pragurile de densitate sau ipotezele privind numărul minim de caractere trebuie acordate la dimensiunea, fontul și dispunerea caracterelor. În contextul autohton, marcat tocmai de o diversitate ridicată a acestor atribute, o asemenea dependență devine o sursă de fragilitate, descriptorul fiind sensibil la formatele atipice sau la plăcuțele cu un număr redus de caractere.

În contrast, metodele bazate pe contururi tratează concentrarea de tranziții doar ca pe un indiciu de salianță, lăsând decizia finală în seama unui invariant geometric independent de conținutul intern al plăcuței: forma dreptunghiulară cu un raport de aspect cunoscut. Întrucât acest criteriu nu depinde de modul în care sunt distribuite caracterele, ci de anvelopa regiunii, el rămâne stabil indiferent de fontul, dimensiunea sau numărul caracterelor, oferind o capacitate de generalizare superioară pentru spectrul larg de formate vizat în prezenta lucrare.

Pornind de la aceste considerente, etapa de localizare a fost implementată în clasa `LPExtraction`, parametrizată prin raportul de aspect admis pentru o plăcuță (`lp_aspect_ratio`), aria minimă a unei regiuni candidate (`lp_min_size`), o regiune de interes opțională (`lp_roi`) și numărul de candidați returnați (`top_k`). Fluxul de procesare urmează etapele clasice ale abordării pe contururi, descrise în capitolul 2, adaptate contextului autohton.

Preprocesarea începe, dacă este definită, cu decuparea regiunii de interes, urmată de conversia în tonuri de gri și de aplicarea unui filtru median pentru atenuarea zgomotului fără estomparea muchiilor. Asupra imaginii filtrate se aplică operatorul Sobel pe direcția orizontală, evidențiind exclusiv muchiile verticale generate de caractere. Renunțarea la muchiile orizontale urmează observația din [6], conform căreia acestea favorizează identificarea eronată a unor elemente precum grilajul radiatorului drept plăcuțe.

Rezultatul este binarizat prin metoda Otsu în variantă inversată, astfel încât regiunile cu densitate ridicată de muchii să devină prim-plan alb, formatul așteptat de funcția de extragere a conturilor. Asupra imaginii binare se aplică o operație morfologică de deschidere, pentru eliminarea structurilor izolate, urmată de o dilatare cu un element structurant orizontal a cărui lățime este scalată proporțional cu lățimea imaginii. Această dilatare are rolul de a contopi muchiile verticale individuale ale caracterelor într-o regiune compactă, corespunzătoare plăcuței.

Din imaginea astfel obținută se extrag contururile externe, iar dreptunghiurile lor de încadrare sunt filtrate pe baza raportului de aspect și a ariei minime, eliminând regiunile a căror geometrie nu corespunde unei plăcuțe.

```
greyscale = cv.cvtColor(array, cv.COLOR_BGR2GRAY)
```

```
median = self._helper.median_filter(array=greyscale)
sobel = self.sobel(median)
thresh = self._helper.otsu_thresholding(sobel)
thresh = cv.bitwise_not(thresh)
open_image = self._helper.opening(thresh)
dilated_image = self._helper.dilation(open_image)
bounding_box = self._helper.find_countours(dilated_image,
                                             self._lp_aspect_ratio,
                                             self._lp_min_size)
```

O decizie de proiectare proprie vizează reasamblarea fragmentelor de plăcuță. În practică, o plăcuță se fragmentează frecvent în etapa de extragere a conturilor: spațiul central dintr-o plăcuță auto o divizează pe orizontală, iar plăcuțele de motocicletă, dispuse pe două rânduri, se divizează pe verticală. Reasamblarea acestor fragmente nu a fost realizată în etapa morfologică, deoarece o dilatare suficient de agresivă pentru a le reuni ar risca să unească plăcuța cu regiuni adiacente precum bara de protecție sau grilajul. În schimb, reunirea se realizează ulterior, la nivelul dreptunghiurilor de încadrare, prin metoda `merge_split_regions`: fragmentele apropiate spațial și aliniate (pe aceeași linie de bază, cu înălțimi similare, sau pe aceeași coloană, cu lățimi similare) sunt grupate printr-o structură *union-find* și înlocuite cu reuniunea lor. Un grup a cărui reuniune produce un raport de aspect implauzibil este tratat ca o uniune accidentală, caz în care fragmentele originale sunt păstrate.

În final, candidații rămași sunt ordonați după densitatea de muchii din interiorul lor, iar primii `top_k` sunt returnați. Returnarea mai multor candidați, în locul unuia singur, constituie o decizie de proiectare deliberată: ea permite etapei următoare de segmentare să recupereze eventualele erori de localizare, alegând candidatul care conduce la cea mai plauzibilă segmentare.

Segmentarea caracterelor

Etapa de segmentare primește candidații de plăcuță produși de etapa de localizare și are ca scop izolarea fiecărui caracter într-o imagine distinctă. Dintre tehnicile prezentate în capitolul 3, a fost adoptată analiza proiecțiilor, combinată cu utilizarea cunoștințelor apriori privind aspectul caracterelor. Această alegere a fost preferată unei abordări bazate exclusiv pe analiza conturilor caracterelor, care prezintă o sensibilitate ridicată la zgomot și la fenomenele de fuziune sau fragmentare a caracterelor, depinzând totodată de praguri dificil de calibrat. Proiecțiile oferă, în schimb, o reprezentare unidimensională robustă a distribuției pixelilor de prim-plan, mai puțin sensibilă la variații locale.

Pipeline-ul de segmentare începe cu binarizarea decupajului prin metoda Otsu inversată, obținând o imagine în care caracterele constituie prim-planul alb. Asupra acesteia se calculează *proiecția orizontală*, prin însumarea pixelilor activi de-a lungul liniilor, pentru a izola banda sau benzile de text. Această etapă este esențială pentru tratarea uniformă a plăcuțelor dispuse pe un singur rând și a celor de motocicletă, dispuse pe două rânduri, întrucât separarea pe benzi normalizează proporțiile caracterelor în raport cu sub-segmentul care le conține, conform argumentului din capitolul 3.

Pentru fiecare bandă de text se calculează apoi *proiecția verticală*, prin însumarea pixelilor activi de-a lungul coloanelor. În profilul rezultat, caracterele apar drept regiuni de densitate ridicată (vârfuri), separate de spațiile dintre ele, care corespund unor regiuni de densitate scăzută (văi). Granițele caracterelor se obțin prin extragerea regiunilor-vârf, folosind un prag relativ la valoarea maximă a histogramei (metoda `extract_peak_regions`): o regiune este reținută atunci când densitatea sa depășește o fracțiune din maxim, iar lățimea sa este suficient de mare pentru a nu fi un artefact.

```
otsu = self._helper.otsu_thresholding(greyscale)
inverted = 255 - otsu
horizontal_projections = self._projection(inverted, axis=1)
horizontal_lines = self.extract_peak_regions(horizontal_projections)
for line in horizontal_lines:
    vertical_projection = self._projection(text_line, axis=0)
    character_coords = self.extract_peak_regions(vertical_projectio
```

Deoarece etapa de localizare poate furniza mai mulți candidați de plăcuță, segmentarea este rulată pe fiecare dintre aceștia, iar setul de caractere reținut este cel care obține scorul maxim conform unei funcții de evaluare (`_score`). Acest cuplaj prin parametrul `top_k` transformă etapa de segmentare într-un mecanism de recuperare a erorilor de localizare: un candidat incorect (de exemplu o regiune de grilaj reținută eronat) va produce o segmentare cu un scor scăzut și va fi astfel respins în favoarea candidatului corect.

Funcția de evaluare valorifică o cunoștință apriori esențială: numărul de caractere al unei plăcuțe valide este cunoscut. Scorul integrează, în consecință, mai multe componente: abaterea numărului de vârfuri detectate față de numărul așteptat de caractere (în jur de șapte simboluri pentru formatul autohton), regularitatea lățimilor vârfurilor (caracterele unei plăcuțe au lățimi comparabile), regularitatea spațiilor dintre vârfuri (spațiere uniformă) și contrastul dintre vârfuri și văi (văi adânci indică o separare curată a caracterelor). O segmentare care satisface aceste constrângeri

primește un scor ridicat, întrucât corespunde tiparului geometric al unei plăcuțe reale.

Profilul de proiecție este consolidat printr-o analiză de tip *crossing-count*, în care, pentru fiecare coloană, se numără tranzițiile alb-negru în locul însumării simple a pixelilor de prim-plan. O coloană care traversează interiorul unui caracter prezintă numeroase tranziții, în timp ce o coloană dintr-un spațiu între caractere prezintă puține; spre deosebire de proiecția simplă, acest profil este mai robust la variațiile grosimii trăsăturii caracterelor. În implementarea propusă (metoda `_crossing_contrast`), contrastul dintre numărul mediu de tranziții din interiorul casetelor de caracter și cel din spațiile dintre ele este adăugat ca termen suplimentar în funcția de evaluare `_score`, completând regularitățile geometrice descrise mai sus.

Recunoașterea caracterelor

Ultima etapă a fluxului de procesare atribuie fiecărui caracter segmentat o etichetă, dintr-un alfabet de cifre și litere. Pentru această sarcină a fost ales un clasificator de tip *Support Vector Machine* (SVM), ale cărui fundamente teoretice au fost prezentate în capitolul 4.

Fiecare caracter este adus la o reprezentare uniformă, independentă de dimensiunea sa originală: imaginea este redimensionată la o rezoluție fixă de 28×28 pixeli, valorile intensității sunt normalizate în intervalul $[0, 1]$, iar matricea rezultată este liniarizată într-un vector de trăsături de 784 de componente. Această reprezentare constituie intrarea clasificatorului.

Clasificatorul folosește un nucleu de tip funcție cu bază radială (RBF), a cărui capacitate de a separa clase neliniar separabile a fost discutată în capitolul 4. Hyperparametrul de regularizare C controlează compromisul dintre maximizarea marginii și penalizarea erorilor de clasificare, iar parametrul γ determină raza de influență a fiecărui vector suport. Pentru a asigura stabilitatea statistică a antrenării, clasele cu un număr insuficient de mostre sunt eliminate din setul de date, pe baza unui prag minim (`MIN_SAMPLES_PER_CLASS`), aspect detaliat în secțiunea dedicată construirii setului de date.

5.3.3 Nivelul de aplicație

Modulul de logică a aplicației constituie, alături de modulul de intrare, una dintre marginile extensibile ale sistemului. Rolul său este de a consuma rezultatul produs de fluxul de procesare (numărul de înmatriculare recunoscut, însoțit de meta-date) și de a declanșa acțiunea corespunzătoare contextului de execuție. Întrucât această acțiune variază semnificativ de la o aplicație la alta, comportamentul este abstractizat printr-o interfață, urmând aceeași strategie ca în cazul modulului de

intrare. Astfel, nucleul rămâne neutru față de scopul concret al aplicației, iar integrarea într-un mediu nou se reduce la furnizarea unei implementări particulare a acestei interfețe.

Sistem embedded

În scenariul unui sistem integrat, modulul de logică a aplicației acționează direct asupra mediului fizic. O implementare reprezentativă este cea a unui punct de control al accesului, în care recunoașterea unui număr de înmatriculare aflat într-o listă de autorizare declanșează ridicarea unei bariere auto prin acționarea echipamentului hardware corespunzător. Aceeași implementare poate emite, în paralel, notificări automate, de exemplu prin e-mail, către un operator sau către un sistem de evidență, atât pentru accesul autorizat, cât și pentru tentativele neautorizate. Caracterul abstract al interfeței permite combinarea liberă a unor astfel de acțiuni, fără modificarea nucleului.

Aplicație demo

Pentru aplicația demonstrativă, modulul de logică a aplicației nu acționează asupra unui echipament fizic, ci expune rezultatele recunoașterii către un client extern. Implementarea se bazează pe biblioteca FastAPI, care oferă infrastructura unui server web, și pe protocolul WebSocket, ales pentru capacitatea sa de a menține o conexiune bidirecțională persistentă. Prin intermediul acestei conexiuni, rezultatele procesării sunt transmise în timp real către interfața clientului, pe măsură ce sunt produse de fluxul de procesare, fără a necesita interogări repetate din partea acestuia. Această implementare ilustrează modul în care același nucleu de procesare poate deservi un context complet diferit de cel al sistemului integrat, prin simpla substituie a implementării modulului de logică.

5.4 Seturi de date

Sistemul IORA operează cu două seturi de date distincte: un set de evaluare, utilizat pentru măsurarea performanței end-to-end a pipeline-ului, și un set de antrenare, destinat construirii clasificatorului SVM de caractere. Ambele gravitează în jurul aceluiași corpus de imagini cu plăcuțe de înmatriculare obținut prin platforma Roboflow [28], completat, în cazul setului de evaluare, de o componentă românească colectată manual, iar în cazul setului de antrenare, de un set de date dedicat caracterelor individuale cu fonturi variate.

5.4.1 Setul de evaluare

Pentru a determina robustețea sistemului și capacitatea sa de adaptare la diverse formate ale numărului de înmatriculare a fost necesară crearea unui set de date mixt, în care sunt prezente atât autovehicule de pe teritoriul României, cât și vehicule din alte regiuni ale globului. Pentru prima componentă a setului de date în momentul de față nu există o sursă general valabilă și deschisă publicului larg, așadar imaginile reprezentative acestei părți au fost colectate manual de pe platforme de comerț auto online. Această procedură de salvare și utilizare a imaginilor este conformă cu reglementările GDPR, întrucât fotografiile au fost publicate în mod voluntar de către vânzătorii pe platforme cu acces liber, fiind astfel destinate publicului larg, iar prelucrarea lor s-a realizat strict în scop de cercetare și evaluare a sistemului. Pentru componenta internațională, a fost utilizat setul de date public obținut prin intermediul platformei Roboflow [28].

Normalizarea seturilor de date

În contextul sistemelor de recunoaștere automată a numerelor de înmatriculare puse în funcțiune în cadrul parcărilor condițiile de execuție precum poziția numărului de înmatriculare, distanța și raportul dimensiunilor, unghiul de captare și distanța focală sunt bine cunoscute, permitând valorificarea lor în procesul de localizare.

Pentru a reproduce aceste condiții, a fost necesară normalizarea setului de date. În cazul imaginilor obținute în mod manual, acest procedeu a presupus selectarea autovehiculelor care prezentau un număr de înmatriculare vizibil, urmată de o decupare pentru a reduce zgomotul de fundal și a oferi o distanță focală relativ constantă. Un exemplu al procesului poate fi observat în figura 5.4, unde înainte de normalizare erau prezente elemente de zgomot agresive atât în parbrizul mașinii cât și în spațiul înconjurător.



(a) Înainte de normalizare



(b) După normalizare

Figura 5.4: Un exemplu de imagine alterată pentru a reduce zgomotul de fundal și normaliza unghiul de captură

Pentru componenta internațională a setului de date procesul a fost unul mai rapid, fiind necesară doar selectarea imaginilor corespunzătoare criteriilor menționate anterior. În figura 5.5 se poate observa un exemplu de imagine corespunzătoare pusă în paralel cu o imagine care nu adera la standardele impuse.



(a) Imagine corespunzătoare



(b) Imagine necorespunzătoare

Figura 5.5: Paralelă între o imagine conformă și una neconformă

Crearea sursei de adevăr a setului de date

Procedeul descris anterior constituie, însă, doar primul pas în construirea unui set de date potrivit strategiei de procesare propuse de această teză. Este în continuare necesară o sursă de adevăr uniformă pe componentele locală și internațională. Această secțiune prezintă pașii luați în completarea imaginilor cu informații relevante și conforme cu realitatea, care să reprezinte reperul față de care vor fi evaluate predicțiile generate de *pipeline*.

Prima etapă este reprezentată de adnotarea imaginilor cu numărul de înmatriculare afișat în acestea, sarcină realizată manual, prin deschiderea lor rând pe rând, urmată de adăugarea valorilor într-un fișier csv dedicat. Acest lucru va fi utilizat atât în numărarea de predicții exacte, cât și în măsurarea mediei distanței Levenshtein. Pentru a permite analizarea performanței componente de localizare, a fost dezvoltat un utilitar software dedicat extragerii chenarelor ce încadrează numerele de înmatriculare (*bounding boxes*). Această aplicație cu interfață grafică facilitează delimitarea manuală a regiunii de interes, permițând utilizatorului să definească chenarul de încadrare prin marcarea coordonatelor aferente colțurilor din stânga-sus și dreapta-jos ale plăcuței.

5.4.2 Setul de antrenare a clasificatorului SVM

Antrenarea clasificatorului de caractere necesită un set de date format din imagini de caractere individuale, fiecare asociată cu eticheta sa. Construirea acestui set a urmat o strategie hibridă pe două surse: caractere generate automat prin aplica-

rea pipeline-ului de procesare asupra imaginilor de antrenare din setul Roboflow de plăcuțe, urmate de etichetare manuală, și un set extern de caractere individuale cu fonturi variate, adnotat în format Pascal VOC. Combinarea celor două surse urmărește, pe de o parte, alinierea distribuției datelor de antrenare la caracteristicile particulare ale caracterelor segmentate de pipeline-ul propriu și, pe de altă parte, asigurarea unui volum și a unei diversități tipografice suficiente.

Generarea automată prin fluxul de procesare

Prima sursă de date este generată de însuși sistemul, prin scriptul `generate_dataset.py`. Acesta aplică, asupra imaginilor de antrenare din setul Roboflow de plăcuțe, etapele de localizare și de segmentare descrise anterior, salvând caracterele segmentate drept imagini individuale, încă neetichetate. Avantajul acestei surse este că mostrele rezultate reflectă fidel caracteristicile caracterelor așa cum sunt produse de pipeline-ul propriu — inclusiv eventualele artefacte de segmentare —, ceea ce aliniază distribuția datelor de antrenare la cea a datelor întâlnite la rulare. Mostrele astfel obținute necesită o etapă ulterioară de etichetare.

Sursă externă de caractere

A doua sursă de date o constituie un set de date de caractere individuale, disponibil prin platforma Roboflow, care acoperă o varietate mai largă de fonturi față de cele prezente în setul de plăcuțe. Adnotările sunt furnizate în format Pascal VOC (fișiere XML). Scriptul `parse_characters.py` parcurge perechile imagine-adnotare, extrage din fiecare fișier XML etichetele caracterelor și dreptunghiurile lor de încadrare, decupează regiunile corespunzătoare din imagine și le salvează în directoare organizate pe clase (câte un director per caracter). Rezultatul este un set de mostre curat etichetate, obținut fără efort manual, care completează diversitatea tipografică a primei surse.

Etichetarea manuală asistată

Etichetarea mostrelor generate automat este realizată prin scriptul `label_dataset.py`, care oferă o interfață minimală de etichetare asistată, construită cu ajutorul bibliotecii OpenCV. Fiecare imagine neetichetată este afișată succesiv, iar operatorul atribuie eticheta prin apăsarea tastei corespunzătoare caracterului; imaginea este mutată automat în directorul clasei respective. Tasta dedicată permite marcarea mostrelor nevalide (caractere nerecunoscute sau segmentate eronat) prin mutarea lor într-un director special, în timp ce o altă tastă permite ignorarea unei mostre. Acest mecanism reduce semnificativ efortul de etichetare, transformându-l într-o succesiune rapidă de apăsări de taste.

Filtrarea și igienizarea claselor

Înainte de antrenare, setul de date consolidat este supus unei etape de filtrare și igienizare, realizată la încărcare. Sunt eliminate clasele a căror etichetă nu corespunde unui singur caracter alfanumeric din setul ASCII, precum și clasa rezervată mostrelor nevalide. În plus, clasele care nu ating un număr minim de mostre (`MIN_SAMPLES_PER_CLASS`) sunt excluse, întrucât un număr prea redus de exemple nu permite o estimare statistică fiabilă și ar afecta atât antrenarea, cât și evaluarea echilibrată a clasificatorului. Această etapă asigură un set de date curat și suficient de echilibrat pentru etapa de antrenare.

5.5 Testare și metrici

Evaluarea sistemului urmărește două niveluri complementare: performanța componentei de recunoaștere a caracterelor, considerată izolat, și performanța întregului lanț de procesare, măsurată de la imaginea de intrare până la șirul de caractere recunoscut.

Pentru evaluarea clasificatorului de caractere, setul de date este împărțit într-un subset de antrenare și unul de testare, în proporție de 80%, respectiv 20%. Împărțirea este stratificată, astfel încât distribuția claselor să fie păstrată în ambele subseturi, condiție importantă în prezența unui set dezechilibrat. Pentru a asigura reproductibilitatea rezultatelor, sămânța generatorului de numere aleatoare este fixată.

Metricile raportate pentru clasificator sunt acuratețea globală și raportul de clasificare, care cuprinde, pentru fiecare clasă, precizia (*precision*), rata de regăsire (*recall*) și scorul F_1 . Aceste metrici sunt completate de matricea de confuzie, utilă pentru identificarea perechilor de caractere frecvent confundate (de exemplu cifra 0 și litera O, sau cifra 1 și litera I).

Pe lângă evaluarea izolată a clasificatorului, sistemul este evaluat și la nivel de etapă: rata de localizare corectă a plăcuței, rata de segmentare corectă a caracterelor și, în final, metrica integrată (*end-to-end*), reprezentată de proporția plăcuțelor citite corect în întregime. Această din urmă metrică reflectă cel mai fidel performanța percepută a sistemului, întrucât o singură eroare la nivel de caracter invalidează întregul rezultat.

Rezultatele agregate obținute prin rularea scripturilor `src/evaluate.py` (pentru metricile de nivel etapă și *end-to-end*) și `src/pipeline/svm.py` (pentru acuratețea clasificatorului) sunt sintetizate în tabelul 5.1. Evaluarea localizării utilizează setul `data/test/`, format din 26 de imagini de autovehicule cu adnotări la nivel de plăcuță, în timp ce evaluarea segmentării și a recunoașterii *end-to-end* este realizată pe setul `data/LP-characters/`, alcătuit din 209 plăcuțe deja decupate, cu

adnotări la nivel de caracter.

Metrică	Raport	Valoare
Acuratețe clasificator caractere	—	0,909
Rată de localizare corectă ($\text{IoU} \geq 0,5$)	8/26	0,308
Rată de segmentare corectă (potrivire la nivel de număr de caractere)	7/209	0,033
Rată de recunoaștere end-to-end (egalitate exactă)	1/209	0,005

Tabela 5.1: Rezultatele evaluării sistemului IORA pe seturile adnotate disponibile.

Raportul de clasificare per clasă (tabelul 5.2) detaliază precizia, rata de regăsire și scorul F_1 pentru fiecare simbol din alfabet, iar tabelul 5.3 sintetizează principalele confuzii identificate prin matricea de confuzie.

Clasă	Precizie	Regăsire	F_1	Mostre
0	0,83	0,88	0,86	17
1	0,89	1,00	0,94	16
2	0,94	1,00	0,97	16
3	0,94	1,00	0,97	17
4	1,00	0,92	0,96	13
5	0,97	0,97	0,97	30
6	0,95	1,00	0,97	18
7	1,00	1,00	1,00	16
8	0,92	0,85	0,88	13
9	1,00	1,00	1,00	19
A	0,80	1,00	0,89	20
B	0,83	0,91	0,87	11
C	1,00	0,91	0,95	11
D	0,62	0,71	0,67	7
E	1,00	1,00	1,00	5
F	1,00	1,00	1,00	12
G	1,00	0,88	0,93	8
H	0,87	0,91	0,89	22
J	0,67	0,67	0,67	3
K	1,00	0,92	0,96	13
L	0,79	0,92	0,85	12
M	0,79	0,94	0,86	16
N	1,00	0,67	0,80	9
P	1,00	0,60	0,75	5
R	0,83	0,50	0,62	10
S	1,00	0,40	0,57	5
T	0,93	0,93	0,93	14
U	1,00	0,33	0,50	3
Medie macro	0,91	0,85	0,87	361
Medie ponderată	0,92	0,91	0,90	361

Tabela 5.2: Raport de clasificare per clasă al clasificatorului SVM pe subsetul de testare (361 de mostre, 28 de clase; kernel=rbf, C=10, gamma='scale').

Clasă reală	Clasă prezisă	Erori	Notă
D	0	2	similitudine vizuală
N	M	2	similitudine vizuală
R	A	2	similitudine vizuală
R	1, H, J	1 fiecare	suport redus (10 mostre)
S	5, 6, B	1 fiecare	suport redus (5 mostre)
U	B, M	1 fiecare	suport redus (3 mostre)
0	D, H	1 fiecare	
8	0, H	1 fiecare	

Tabela 5.3: Principalele confuzii ale clasificatorului SVM: perechi real–prezis cu cel puțin o eroare, extrase din matricea de confuzie.

Acuratețea clasificatorului, raportată la 90,9%, confirmă viabilitatea reprezentării 28×28 liniarizate, combinată cu nucleul RBF. Discrepanța dintre această valoare și rata de recunoaștere end-to-end indică faptul că majoritatea erorilor sistemului provin din etapele de localizare și de segmentare, propagându-se ulterior asupra rezultatului final. Tocmai această observație motivează studiile de ablație din secțiunea următoare.

5.6 Experimente

Dincolo de metricile agregate, comportamentul sistemului este analizat printr-o serie de experimente menite să evidențieze contribuția fiecărei decizii de proiectare și reziliența sistemului în raport cu factorii de dificultate identificați în introducerea lucrării.

O primă categorie de experimente o constituie studiile de caz pe factorii de reziliență: condiții de iluminare variabile, formate diferite de plăcuță (auto pe un rând și motocicletă pe două rânduri) și prezența unor reziduuri care obstrucționează parțial caracterele. Aceste studii urmăresc să stabilească limitele de operare ale sistemului și scenariile în care performanța se degradează.

O a doua categorie o reprezintă studiile de ablație, prin care se izolează contribuția componentelor cheie ale fluxului de procesare. Sunt vizate, în special: efectul returnării mai multor candidați de plăcuță (parametrul `top_k`) asupra ratei de recunoaștere, aportul corecției de înclinare (*deskew*) și impactul reasamblării fragmentelor de plăcuță prin `merge_split_regions`. Fiecare componentă este dezactivată pe rând, iar variația metricilor cuantifică aportul său.

În fine, un experiment dedicat compară direct cele două abordări ale etapei de segmentare: varianta bazată pe analiza contururilor și varianta bazată pe proiecții

combinate cu funcția de evaluare. Această comparație are rolul de a justifica, prin rezultate cantitative, alegerea metodei de segmentare adoptate.

Rezultatele celor cinci configurații, obținute prin rularea scriptului `src/experiments.py`, sunt sintetizate în tabelele 5.4 și 5.5. Pentru fiecare variantă se raportează aceleași trei rate (localizare, segmentare, end-to-end) ca în tabelul 5.1, ceea ce permite atribuirea directă a variațiilor observate componentei dezactivate.

Configurație	Localizare	Segmentare	End-to-end
Baseline (<code>top_k=5</code> , <i>deskew</i> și <code>merge_split_regions</code> active)	0,308	0,033	0,005
<code>top_k=1</code>	0,077	0,033	0,005
<i>Deskew</i> dezactivat	0,308	0,033	0,005
<code>merge_split_regions</code> dezactivat	0,269	0,033	0,005

Tabela 5.4: Studii de ablație: contribuția fiecărei componente cheie a etapei de localizare.

Reducerea valorii `top_k` de la 5 la 1 produce cea mai mare scădere a ratei de localizare, de la 30,8% la 7,7%. Acest rezultat confirmă, pe cale cantitativă, decizia de proiectare descrisă în secțiunea 5: returnarea mai multor candidați, în locul unuia singur, permite etapei de segmentare să recupereze erorile de localizare prin alegerea candidatului cu cel mai bun scor. Dezactivarea reasamblării fragmentelor de plăcuță (`merge_split_regions`) determină o scădere mai redusă, de aproximativ 4 puncte procentuale, în timp ce dezactivarea corecției de înclinare (*deskew*) nu afectează metricile pe setul de date utilizat, sugerând că imaginile din `data/test/` prezintă unghiuri de înclinare neglijabile.

Metodă de segmentare	Localizare	Segmentare	End-to-end
Bazată pe proiecții (implementarea propusă)	0,308	0,033	0,005
Bazată pe contururi (variantă moștenită)	0,308	0,072	0,043

Tabela 5.5: Comparație directă între cele două abordări ale etapei de segmentare, pe același set de candidați produși de etapa de localizare.

Comparația dintre cele două metode de segmentare oferă un rezultat aparent contraintuitiv: pe setul de date utilizat, abordarea bazată pe contururi obține o rată end-to-end de 4,3%, comparativ cu 0,5% pentru cea bazată pe proiecții. O analiză posibilă a acestei discrepante este aceea că setul `data/LP-characters/` conține preponderent plăcuțe cu un fundal uniform, în care contururile caracterelor sunt bine separate, scenariu favorabil abordării bazate pe contururi. Avantajele teoretice ale proiecțiilor (robustețea la zgomot și la pragurile dificil de calibrat) s-ar manifesta probabil mai pregnant pe un set mai eterogen, observație care motivează extinderea ulterioară a setului de evaluare.

5.6.1 Îmbunătățiri progresive ale fluxului de recunoaștere

Plecând de la configurația de bază descrisă în secțiunile anterioare, au fost adoptate, succesiv, patru intervenții incrementale care vizează etapele de segmentare, decodare și clasificare. Evaluarea s-a făcut pe setul `data/VALIDATE/` (186 de imagini cu adevăr de referință al șirului de caractere), iar metricile raportate sunt rata end-to-end (egalitate exactă), similaritatea Levenshtein a OCR-ului, acordul la nivel de număr de caractere segmentate și IoU-ul localizării. Tabelul 5.6 sintetizează contribuția cumulativă a fiecărei intervenții.

Configurație	End-to-end	OCR sim.	Acord seg.	IoU
Bază	0,489	0,768	0,809	0,588
+ reconstrucția segmentării	0,597	0,801	0,839	0,588
+ decodare condiționată de format	0,645	0,801	0,839	0,588
+ trăsături HOG pentru SVM	0,667	0,807	0,839	0,588
+ pre-procesare cu conservarea proporțiilor	0,677	0,806	0,839	0,588
+ padding LP și augmentare cu rotații	0,694	0,813	0,843	0,589

Tabela 5.6: Contribuția cumulativă a celor cinci intervenții. Fiecare linie include modificările liniilor anterioare. Cumulat, rata end-to-end a crescut cu 20,5 puncte procentuale ($91 \rightarrow 129$ plăcuțe corecte din 186).

Reconstrucția etapei de segmentare

Înainte de intervenție, decupajul de plăcuță era binarizat direct, iar contururile externe erau filtrate exclusiv prin raportul de aspect w/h , cu un prag inferior de 0,15. Matricea de confuzii a arătat două surse sistematice de erori: aproximativ 250 de caractere lipsă (dominate de $5 \rightarrow \emptyset$ cu 55 de cazuri, $1 \rightarrow \emptyset$ cu 48 de cazuri) și aproximativ 30 de caractere inserate fals ($\emptyset \rightarrow A$, $\emptyset \rightarrow T$, $\emptyset \rightarrow H$). Caracterul 1, cu un raport real w/h între 0,07 și 0,12, era respins în întregime de pragul geometric, iar reziduurile cadrului plăcuței (banda UE, șuruburile de prindere) produceau contururi care imitau caractere reale.

Soluția a constatat în (1) restrângerea decupajului la cea mai mare componentă conexă deschisă din interiorul plăcuței, eliminând cadrul înainte de binarizare; (2) coborârea pragului w/h la 0,08, recuperând caracterul 1; (3) respingerea contururilor care ating marginile laterale ale plăcuței tăiate, anulând patternul de inserții; (4) o trecere de consistență pe înălțimi, care elimină casetele aflate în afara intervalului $[0,55m, 1,45m]$, unde m este înălțimea mediană a supraviețuitorilor; (5) un divizor de blocuri largi care, pentru casete cu $w > 1,2h$, taie la coloana cu cea mai joasă proiecție verticală, recuperând caracterele lipite. Inserțiile au scăzut de la ~ 30 la ~ 5 , iar rata end-to-end a urcat cu 10,8 puncte procentuale.

Au fost respinse explicit aplicarea unei închideri morfologice 3×3 , care, pe plăcuțe de ~ 27 de pixeli înălțime, a unit caracterele adiacente (numărul de contururi crude a scăzut de la 23 la 4 într-un caz observat), și varianta verticală (2, 1), cu același efect în formă atenuată.

Decodare condiționată de format

Analiza confuziilor reziduale a indicat substituții sistematice între clase vizual similare ($4 \rightarrow F$, $D \rightarrow 0$, $Y \rightarrow T$) generate de ambiguitatea internă a SVM-ului. Adevărul de referință al setului `data/VALIDATE/` respectă în 92% din cazuri unul dintre formatele `DDLDDDDD` (8 caractere) sau `DDLDDDD` (7 caractere); aceste formate sunt derivate strict din distribuția setului, întrucât testul nu conține plăcuțe românești.

Modificarea, localizată în metoda `predict` a clasei `CharacterRecognition`, înlocuiește predicția top-1 a SVM-ului cu `decision_function` și restrânge `argmax`-ul fiecărei poziții la setul de caractere admis (cifre pentru toate pozițiile cu excepția poziției 2, literă pentru poziția 2). Pentru lungimi care nu corespund vreunui prior, decodarea revine la varianta neconstrânsă, evitând orice regresie. Acțiunea selectivă a priorului asupra ambiguităților SVM a urcat rata end-to-end cu 4,8 puncte procentuale, fără modificarea similarității Levenshtein (doar ~ 10 predicții de caracter s-au schimbat, dar exact cele care invalidau șirul întreg).

Tot în acest studiu a fost respinsă o margine adițională la decupajul plăcuței, intenționată să recupereze caracterele tăiate la localizare: padding-uri de 2% și 5% au regresat rata end-to-end la 47,3%, respectiv 41,9%. Cauza este că majoritatea decupajelor erau deja corecte (IoU median 0,655), iar adăugarea de margine introducea fragmente de caroserie sau de plăcuțe vecine; pierderea de caractere inițiale se concentrează într-o coadă a distribuției IoU (sub 0,3), nu într-un bias sistematic al localizatorului, pe care un padding uniform nu îl poate corecta.

Trăsături HOG pentru clasificator

Reprezentarea pe 784 de dimensiuni (intensitățile brute ale pixelilor 28×28) a fost înlocuită cu un descriptor HOG configurat peste aceeași fereastră, cu celule de 7×7 , blocuri de 14×14 și un pas de 7 pixeli, producând un vector de 324 de componente. HOG-ul captează distribuția orientărilor muchiilor, în mare măsură invariantă la iluminare și la mici perturbații geometrice — direcția exactă în care reprezentarea pe pixeli bruți era fragilă, întrucât caracterele segmentate la rulare prezintă variații de contrast și de margini față de cele din setul de antrenare.

Modelul a fost re-antrenat fără alte modificări ale scriptului `src/pipeline/svm.py`, atingând o acuratețe de 97% pe subsetul independent. În pipeline-ul integrat, rata end-to-end a urcat cu 2,2 puncte procentuale (la 66,7%). Mai semnificativ, matricea

de confuzii s-a contractat de la 82 la 70 de perechi unice de erori, indicând o concentrare a erorilor reziduale în categorii mai bine definite și mai ușor de atacat în continuare.

Conservarea proporțiilor caracterului

Ultima intervenție vizează o limitare structurală a etapei de pregătire a caracterului: redimensionarea directă la 28×28 deforma raportul de aspect al claselor înguste. Un caracter 1, cu $w/h \approx 0,12$ la segmentare, devenea un pătrat, iar descriptorul HOG raporta o distribuție de orientări similară cu cea a unui I, T sau L. Modificarea, în `preprocess_char`, completează imaginea cu o margine constantă (mediana celor patru colțuri, care aproximează fundalul plăcuței) până la un pătrat, înainte de redimensionarea la 28×28 .

Această intervenție a oferit și o ilustrare a problemei reprezentativității setului de date: acuratețea pe subsetul independent al setului de antrenare a scăzut ușor ($97\% \rightarrow 95,8\%$), în timp ce rata end-to-end a urcat cu 1,1 puncte procentuale (la $67,7\%$). Diferența reflectă faptul că setul `data/labeled/` provine de la o singură rulare a segmenterului și conține preponderent caractere deja aproape pătrate; la rulare, în schimb, pipeline-ul întâlnește o variație mai mare a raporturilor de aspect, pe care padding-ul o normalizează.

Candidați LP padded și augmentare cu rotații a setului de antrenare

Cele două intervenții descrise aici au fost dezvoltate, măsurate și validate împreună, întrucât atacă fenomene complementare: prima țintește bottleneck-ul de localizare evidențiat anterior (tăierea caracterului inițial), iar a doua robustețea clasificatorului la perturbațiile distribuționale dintre setul de antrenare și cel de la rulare. Suma intervențiilor a urcat rata end-to-end cu 1,7 puncte procentuale (la $69,4\%$, 129 plăcuțe corecte din 186).

Primul element vizează producerea de candidați LP suplimentari prin extindere laterală a celor mai bine clasificați doi candidați. În `LPExtraction.extraction_pipeline_with`, după ce ordonarea pe densitate de muchii a produs cei `top_k` candidați, sunt adăugate la listă două variante derivate ale fiecăruia dintre primii doi: o margine agresivă la stânga (+12% din lățime, ținând clipping-ul caracterului inițial) și o margine bilaterală modestă (+5% pe fiecare parte). Aceste variante intră în turneul de scoring al segmentării (`CharacterSegmentation.character_segmentation`), iar funcția `_score` alege automat varianta care produce cea mai plauzibilă segmentare; pe plăcuțele cu localizare deja corectă, varianta originală câștigă; pe plăcuțele cu clipping pe stânga, varianta extinsă câștigă selectiv. Decizia de a nu aplica padding-ul blanket — strategia respinsă în Part 2 a acestei secțiuni — este astfel păstrată, dar

mecanismul ei subiacent este restaurat prin selectarea condiționată per imagine.

A doua intervenție constă într-un script de augmentare (`src/augment_dataset.py`) care, pentru fiecare mostră etichetată din `data/labeled/`, generează patru variante rotite la ± 3 și ± 5 în jurul centrului, cu marginea umplută cu mediana celor patru colțuri (același criteriu folosit în `preprocess_char`). Setul de antrenare crește astfel de la 1862 la 9310 mostre, iar pragul `MIN_SAMPLES_PER_CLASS` începe să admită clase noi (V, W, X, Y, Z, I, O) care erau anterior excluse pentru lipsă de exemplare. SVM-ul este re-antrenat fără alte modificări în `src/pipeline/svm.py`.

Metrică	Înainte (5671681)	După
Rată end-to-end	0,677	0,694
Similaritate Levenshtein OCR	0,806	0,813
Acord la nivel de număr de caractere	0,839	0,843
Localizare (IoU mediu)	0,588	0,589
Perechi unice în matricea de confuzii	75	76

Tabela 5.7: Efectul combinat al padding-ului de candidați LP și al augmentării cu rotații a setului de antrenare. Aportul izolat al fiecărei intervenții nu a fost cuantificat, întrucât patternul vizat (caracter inițial recuperat de padding și clasificat de SVM-ul mai robust) implică ambele componente.

Efectul observat în matricea de confuzii este concentrat pe categoriile de drops dominate de cifre la stânga plăcuței: $5 \rightarrow \emptyset$ a scăzut de la 55 la 50, $1 \rightarrow \emptyset$ de la 48 la 41, $9 \rightarrow \emptyset$ de la 17 la 10, $4 \rightarrow \emptyset$ de la 16 la 9, iar $3 \rightarrow \emptyset$ de la 20 la 15. Suma drops scăzute totalizează aproximativ 36 de caractere, ceea ce este consistent cu mecanismul intervenției (recuperarea unui caracter pe poziția 0 a ~ 30 de plăcuțe). Câștigul net pe end-to-end este însă mai mic (+3 plăcuțe corecte) deoarece o parte din plăcuțele recuperate aveau și alte erori care le mențin clasificate ca greșite, iar padding-ul a introdus, în paralel, aproximativ 6 inserții noi ($\emptyset \rightarrow 1$, $\emptyset \rightarrow L$, $\emptyset \rightarrow 0$) care au degradat un număr similar de plăcuțe anterior corecte.

Experiment respins (acest tur). Un mecanism de re-extracție condiționată de lungimea ieșirii (denumit A2 pe parcursul dezvoltării) a fost implementat și măsurat. Logica: dacă predicția de pe primul pas avea o lungime sub pragul tipic al setului (< 7 caractere), se aplica un padding mai agresiv (+8%, +15% stânga) pe bounding box-ul care produsese segmentarea câștigătoare, iar segmentarea și recunoașterea erau re-rulate; noua predicție era acceptată numai dacă era strict mai lungă decât prima, încadra în lungimea maximă (≤ 8) și conținea predicția originală ca sufix. Pe setul `data/VALIDATE/`, efectul net a fost 0 puncte procentuale end-to-end și $-0,001$ pe similaritatea Levenshtein, motiv pentru care intervenția a fost respinsă. Cauza diagnosticată prin inspecție directă: pe cazurile reziduale (de exem-

plu CarLongPlate264, GT 52Y6490, recuperat ca 2Y6490), caracterul lipsă nu se mai află în nicio regiune accesibilă prin padding — chiar și o margine de 100% din lățimea decupajului nu îl include, întrucât el a fost pierdut deja în etapa de detecție a conturilor. Mecanismul de re-extracție este corect proiectat, dar bottleneck-ul real este în `LPExtraction`, nu în recuperarea ulterioară.

Bilanț

Cumulat, cele cinci intervenții au urcat rata end-to-end de la 48,9% la 69,4%, cu 20,5 puncte procentuale, sau 38 de plăcuțe corecte suplimentare din cele 186 adnotate. Două limitări reziduale rămân vizibile: tăierea caracterului inițial al plăcuței, cauzată în etapa de localizare (vizibilă în patternul $5 \rightarrow \emptyset$, care s-a redus la ~ 50 de cazuri dar nu a putut fi eliminat, întrucât caracterul nu mai ajunge la segmenter), și un set restrâns de substituții vizual ambigue dintre litere sau dintre cifre ($Y \rightarrow M$, $F \rightarrow 6$, $G \rightarrow 4$), pe care priorul de format nu le poate corecta și care reprezintă, probabil, limita reprezentării HOG pe ferestre de 28×28 .

Estimare pe un set omogen. Analiza erorilor reziduale sugerează că rata end-to-end măsurată este influențată în mod direct de eterogenitatea setului `data/VALIDATE/` și ar crește cu aproximativ 10–15 puncte procentuale pe un set omogen, fără a modifica fluxul de procesare. Argumentul este construit prin atribuirea erorilor către surse specifice de variabilitate, iar fiecare sursă ar dispărea sau s-ar atenua într-un scenariu omogen.

În primul rând, priorul de format acoperă 92% din plăcuțele setului (formatele `DDLDDDD` și `DDLDDDD`); cele 15 plăcuțe cu lungimi sau structuri atipice nu beneficiază de corectarea condiționată per poziție, iar substituțiile lor de tip cifră-literă rămân netratate. Pe un set cu un singur format admis, acoperirea priorului ar fi 100%, iar majoritatea acestor 15 plăcuțe ar trece în categoria corectă.

În al doilea rând, tăierea caracterului inițial este consecința unei dispersii ridicate în IoU-ul localizării (raportat la 0,589 mediu) și a faptului că adnotările au fost realizate manual cu o unealtă cu două click-uri (`src/bb_writer.py`), ceea ce introduce zgomot de încadrare în adevărul de referință în sine. Pe un set cu plăcuțe imagistic uniforme, raportul plăcuță-cadru și unghiul de captură ar fi constante, iar dispersia IoU s-ar reduce semnificativ; estimarea pe baza celor ~ 30 de plăcuțe afectate sugerează recuperarea a 20–25 dintre ele.

În al treilea rând, clasele rare (`V`, `W`, `X`, `Y`, `Z`, `I`, `O`) au depășit pragul minim de antrenare doar prin augmentarea cu rotații, ceea ce indică reprezentativitate marginală. Un set omogen ar avea fie un alfabet activ mai restrâns (eliminând clasele rare din spațiul de decizie), fie un volum suficient de mostre per clasă, ambele variante reducând substituțiile literă-literă care reprezintă restul erorilor.

Cumularea acestor trei surse, fiecare cu contribuție independentă, conduce la o estimare conservativă de 80–84% end-to-end pe un astfel de set, fără a modifica arhitectura clasică folosită aici. Această estimare nu este o extrapolare optimistă: ea presupune doar uniformizarea sursei de date, nu o îmbunătățire a metodelor, și se sprijină pe descompunerea cuantificată a erorilor reziduale prezentată în secțiunile anterioare.

Capitolul 6

Concluzii

Concluzii ...

Bibliografie

- [1] Jithmi Shashirangana, Heshan Padmasiri, Dulani Meedeniya, and Charith Perera. Automated license plate recognition: a survey on methods and techniques. *IEEE Access*, 9:11203–11225, 2020.
- [2] Xifan Shi, Weizhong Zhao, and Yonghang Shen. Automatic license plate recognition system based on color image processing. In *International conference on computational science and its applications*, pages 1159–1168. Springer, 2005.
- [3] Shyang-Lih Chang, Li-Shien Chen, Yun-Chung Chung, and Sei-Wan Chen. Automatic license plate recognition. *IEEE transactions on intelligent transportation systems*, 5(1):42–53, 2004.
- [4] Lin Luo, Hao Sun, Weiping Zhou, and Limin Luo. An efficient method of license plate location. In *2009 First International Conference on Information Science and Engineering*, pages 770–773. IEEE, 2009.
- [5] V Karthikeyan and VJ Vijayalakshmi. Localization of license plate using morphological operations. *arXiv preprint arXiv:1402.5623*, 2014.
- [6] Abbas M Al-Ghaili, Syamsiah Mashohor, Alyani Ismail, and Abdul Rahman Ramli. A new vertical edge detection algorithm and its application. In *2008 international conference on computer engineering & systems*, pages 204–209. IEEE, 2008.
- [7] Tran Duc Duan, TL Hong Du, Tran Vinh Phuoc, and Nguyen Viet Hoang. Building an automatic vehicle license plate recognition system. In *Proc. Int. Conf. Comput. Sci. RIVF*, volume 1, pages 59–63, 2005.
- [8] R Bremananth, A Chitra, Vivek Seetharaman, and Vidya Sudhan L Nathan. A robust video based license plate recognition system. In *Proceedings of 2005 International Conference on Intelligent Sensing and Information Processing*, 2005., pages 175–180. IEEE, 2005.
- [9] Christos Nikolaos E Anagnostopoulos, Ioannis E Anagnostopoulos, Vassilis Loumos, and Eleftherios Kayafas. A license plate-recognition algorithm for

- intelligent transportation system applications. *IEEE Transactions on Intelligent transportation systems*, 7(3):377–392, 2006.
- [10] Kenji Kanayama, Yoshimasa Fujikawa, Koichi Fujimoto, and Masanobu Horino. Development of vehicle-license number recognition system using real-time image processing and its application to travel-time measurement. In *[1991 Proceedings] 41st IEEE Vehicular Technology Conference*, pages 798–804. IEEE, 1991.
- [11] Zhang Sanyuan, Zhang Mingli, and Ye Xiuzi. Car plate character extraction under complicated environment. In *2004 IEEE International Conference on Systems, Man and Cybernetics (IEEE Cat. No. 04CH37583)*, volume 5, pages 4722–4726. IEEE, 2004.
- [12] Abdulkerim Capar and Muhittin Gokmen. Concurrent segmentation and recognition with shape-driven fast marching methods. In *18th International Conference on Pattern Recognition (ICPR'06)*, volume 1, pages 155–158. IEEE, 2006.
- [13] James A Sethian. A fast marching level set method for monotonically advancing fronts. *Proceedings of the National Academy of Sciences*, 93(4):1591–1595, 1996.
- [14] Muhammad Sarfraz, Mohammed Jameel Ahmed, and Syed A Ghazi. Saudi arabian license plate recognition system. In *2003 International conference on geometric modeling and graphics, 2003. Proceedings*, pages 36–41. IEEE, 2003.
- [15] Eun Ryung Lee, Pyeoung Kee Kim, and Hang Joon Kim. Automatic recognition of a car license plate using color image processing. In *Proceedings of 1st international conference on image processing*, volume 2, pages 301–305. IEEE, 1994.
- [16] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386, 1958.
- [17] Vladimir N Vapnik. Pattern recognition using generalized portrait method. *Automation and remote control*, 24(6):774–780, 1963.
- [18] Vladimir N Vapnik. A note on one class of perceptrons. *Automat. Rem. Control*, 25:821–837, 1964.
- [19] Vladimir N Vapnik and A Ya Chervonenkis. On the uniform convergence of relative frequencies of events to their probabilities. In *Measures of complexity: festschrift for alexey chervonenkis*, pages 11–30. Springer, 2015.
- [20] Nello Cristianini and John Shawe-Taylor. *An Introduction to Support Vector Machines and Other Kernel-based Learning Methods*. Cambridge University Press, 2000.

- [21] Leslie G Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984.
- [22] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995.
- [23] Bernhard E Boser, Isabelle M Guyon, and Vladimir N Vapnik. A training algorithm for optimal margin classifiers. In *Proceedings of the fifth annual workshop on Computational learning theory*, pages 144–152, 1992.
- [24] Anil K Jain, Jianchang Mao, and K Moidin Mohiuddin. Artificial neural networks: A tutorial. *Computer*, 29(3):31–44, 1996.
- [25] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feed-forward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.
- [26] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [27] *CODE COMPLETE 2nd Edition*. WP Publishers & Distributors Pvt Limited, 2004.
- [28] Roboflow Universe Projects. License plate recognition dataset. <https://universe.roboflow.com/roboflow-universe-projects/license-plate-recognition-rxg4e>, 2022. accesat iunie 2026.